



Docker



Agenda



01. Introduction

What is Docker ?

Architecture Docker ?

Docker Registry ?

02. Linux Basic

Command Linux basic

03. Docker Volume and Docker Network

What is Docker volume ?

04. Dockerfile

What is Dockerfile ?

05. Docker-compose

What is Docker-compose ?

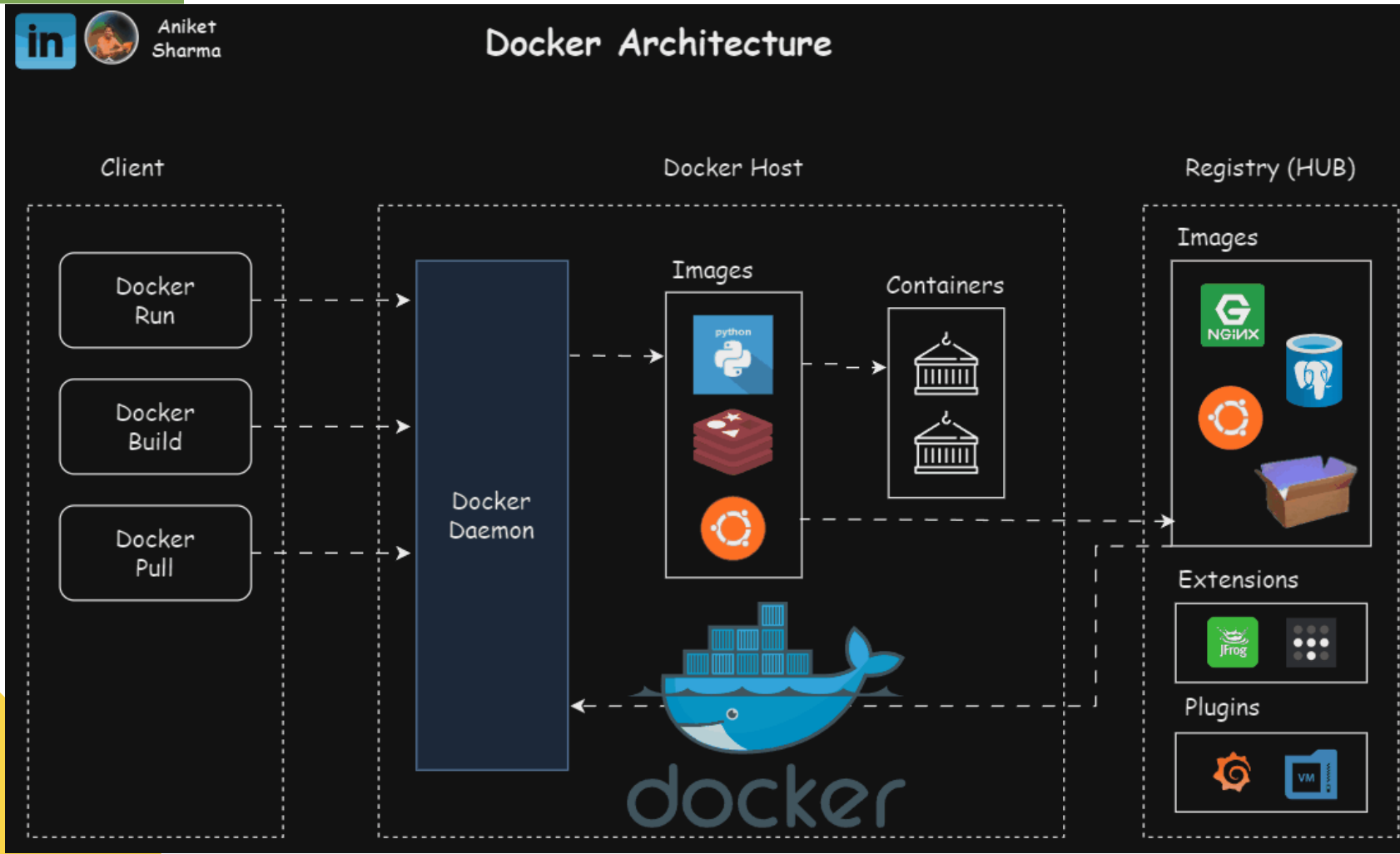
Introduction

Docker คืออะไร

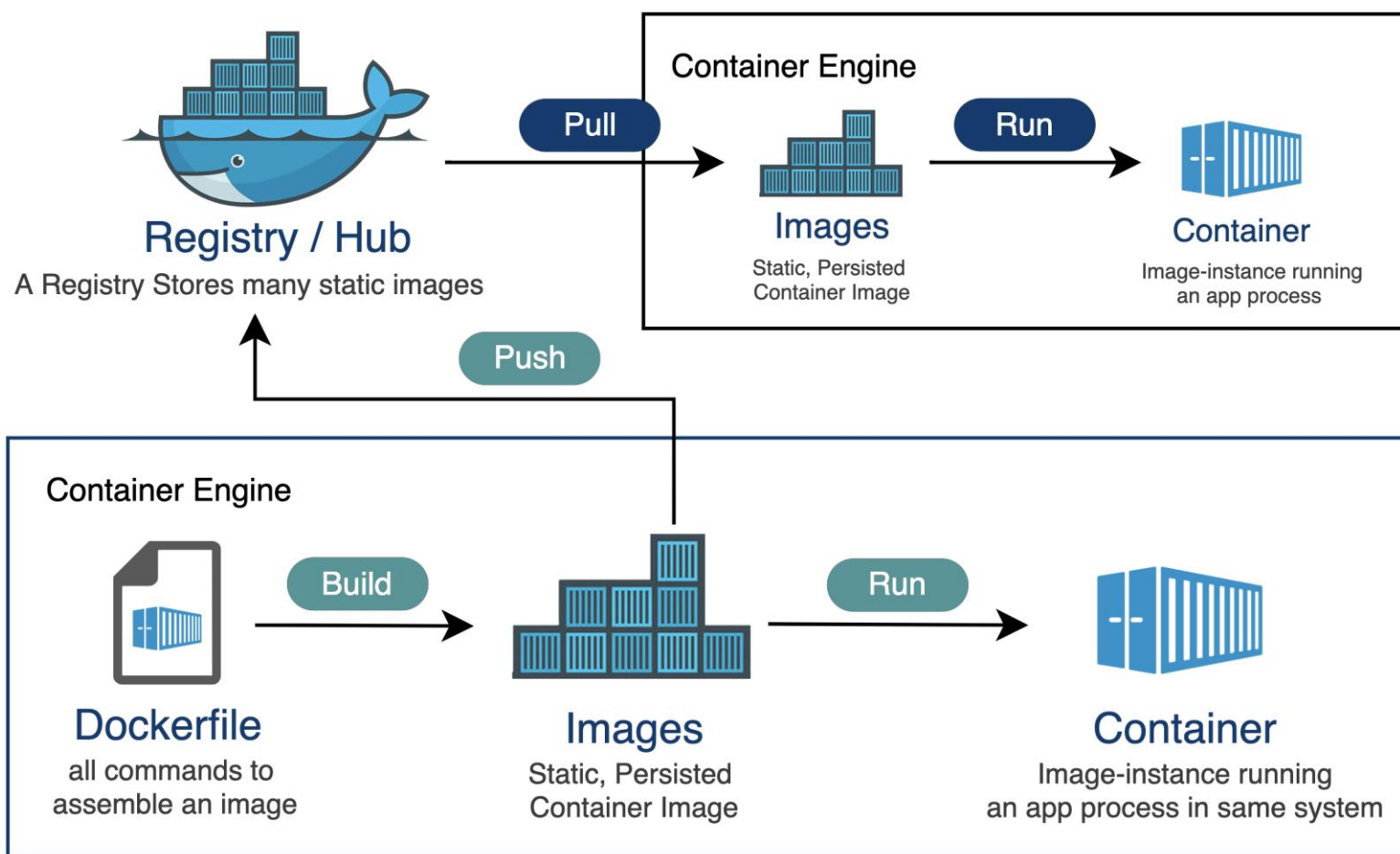
- Docker คือ เครื่องมือแบบ Open-source ที่ช่วยจำลองสภาพแวดล้อม (Environment) ในการรัน Service หรือ Server
- หลักการสร้าง Container เพื่อจัดการกับ Library ต่างๆ อีกทั้งยังมีเครื่องมือในการจัดการในเรื่องของ Version Control เพื่อง่ายต่อการจัดการ และปรับปรุงการบริการ
- ปัจจุบันในโลกของการพัฒนา Software มีรูปแบบการทำงานแบบ Agile ที่เน้นความรวดเร็วในการส่งมอบงานในแต่ละขั้นตอน
- Docker จึงเป็นที่รู้จักในวงกว้างและเริ่มเข้ามามีบทบาทอย่างมากในโลกของการพัฒนา Software สามารถรองรับการติดตั้งใช้งานบนระบบปฏิบัติการที่หลากหลายไม่ว่าจะเป็น Linux, Windows, MAC



Architecture

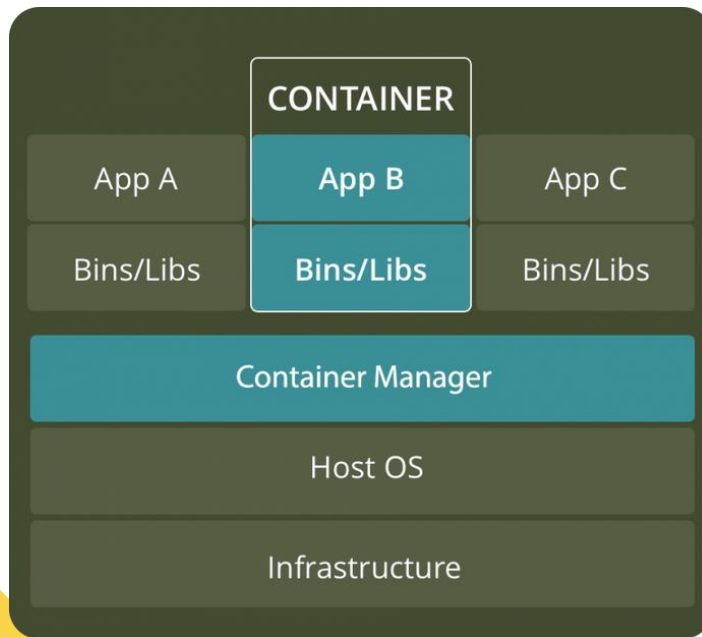


Docker Registry

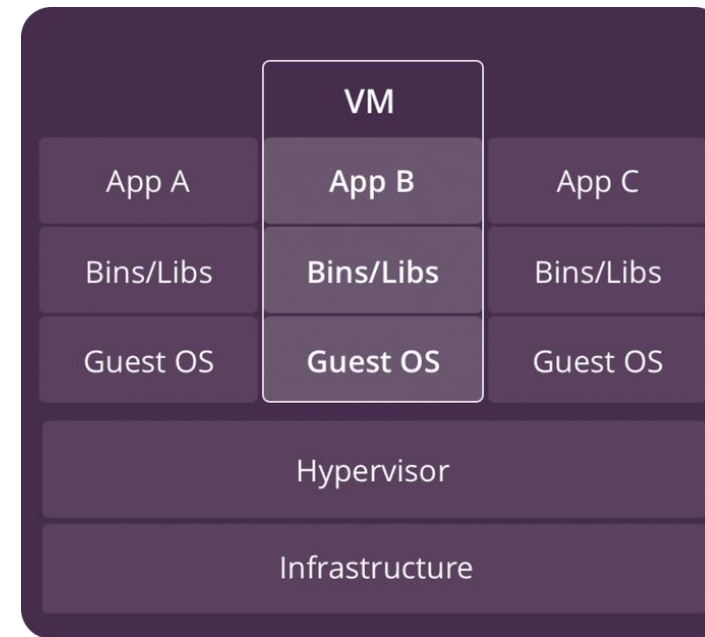


Docker vs VMware

Docker



VMware



ข้อแตกต่างระหว่าง **Docker** และ **VMware**

Docker

- ใช้เวลาไม่กีวินาทีในการ Boot
- จัดการผ่าน Container Engine
- ไม่เปลือง Memory เนื่องจากไม่ต้องใช้ในการสร้างระบบ Virtual
- ระบบไม่ได้แยกการทำงานออกจากกันชัดเจน อาจเกิดความยุ่งยากได้ง่าย
- Deploy Container ทำได้ง่ายเนื่องจาก Container Image สามารถติดตั้งได้ในหลาย ๆ OS

VMware

- ใช้เวลาเป็นนาทีในการ Boot
- จัดการผ่าน Hypervisor
- ใช้ Memory ทุกครั้งเนื่องจาก OS จะต้อง Start ก่อนที่จะใช้งาน Service
- ระบบมีการแยกการทำงานออกจากกัน จัดการปัญหาได้ง่ายกว่า
- Deploy VM มีขั้นตอนที่ต้องเตรียมการมากกว่าเพราะต้องแยก Instance ออกจากกัน

Linux Basic Command

Command

- ls , ls -la , ll คำสั่งที่ใช้ในการ list ดูไฟล์หรือ folder

```
chalersak@Takeit-Youth:~$ ls
11.txt 1111.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack train-jenkins wordpress-old
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ ls -la
total 156
drwxr-xr-x 23 chalersak chalersak 4096 Aug 13 11:06 .
drwxr-xr-x  5 root      root      4096 Aug  2 17:17 ..
-rw-r--r--  1 chalersak chalersak  225 Jul 24 17:39 .angular-config.json
lrwxrwxrwx  1 chalersak chalersak   22 Jun 24 11:06 .aws -> /mnt/c/Users/DELL/.aws
lrwxrwxrwx  1 chalersak chalersak   24 Jun 24 11:06 .azure -> /mnt/c/Users/DELL/.azure
-rw-----  1 chalersak chalersak 30173 Aug 13 11:06 .bash_history
-rw-r--r--  1 chalersak chalersak  220 Jun 24 10:50 .bash_logout
-rw-r--r--  1 chalersak chalersak  4036 Jul 24 17:39 .bashrc
drwx-----  3 chalersak chalersak  4096 Jun 24 16:33 .cache
drwxr-x--x  4 chalersak chalersak  4096 Jul 24 17:23 .config
drwxr-xr-x  5 chalersak chalersak  4096 Aug  9 15:53 .docker
-rw-r--r--  1 chalersak chalersak   250 Jul 11 14:02 .gitconfig
drwxr-xr-x  2 chalersak chalersak  4096 Jun 24 10:50 .landscape
drwxr-xr-x  3 chalersak chalersak  4096 Jun 24 14:41 .local
-rw-r--r--  1 chalersak chalersak    0 Aug 13 11:01 .motd_shown
drwxr-xr-x  6 chalersak chalersak  4096 Jul 31 10:49 .npm
drwxr-xr-x  8 chalersak chalersak  4096 Aug  7 17:56 .nvm
-rw-r--r--  1 chalersak chalersak   807 Jun 24 10:50 .profile
-rw-r--r--  1 chalersak chalersak   66 Jul 11 11:20 .selected_editor
drwx-----  2 chalersak chalersak  4096 Jul  8 11:44 .ssh
-rw-r--r--  1 chalersak chalersak    0 Jun 24 11:03 .sudo_as_admin_successful
-rw-----  1 chalersak chalersak   844 Aug  9 10:54 .viminfo
drwxr-xr-x  3 chalersak chalersak  4096 Jun 24 16:33 .vscode-remote-containers
drwxr-xr-x  5 chalersak chalersak  4096 Jun 24 16:33 .vscode-server
-rw-r--r--  1 chalersak chalersak   230 Jul  8 13:37 .wget-hsts
-rw-r--r--  1 root      root      0 Aug  9 10:51 11.txt
-rwx--x--x  1 root      root      0 Aug  9 10:53 1111.txt
drwxr-xr-x  8 systemd-coredump root      4096 Aug  9 16:34 database
drwxr-xr-x  9 chalersak chalersak  4096 Jul 12 13:53 edoc
drwxr-xr-x  3 chalersak chalersak  4096 Jul 12 11:19 edoc-oracle
drwxr-xr-x  4 chalersak chalersak  4096 Aug  5 16:39 jenkins
```


Linux Basic Command

Command

- touch คำสั่งที่ใช้ในการสร้างไฟล์

λ Cmler

```
chalersak@Takeit-Youth:~/test$ touch 222.txt
chalersak@Takeit-Youth:~/test$ ll
total 8
drwxr-xr-x  2 chalersak chalersak 4096 Apr  4 22:58 ./
drwxr-xr-x 19 chalersak chalersak 4096 Apr  4 22:56 ../
-rw-r--r--  1 root      root          0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalersak chalersak      0 Apr  4 22:58 222.txt
chalersak@Takeit-Youth:~/test$
```

Linux Basic Command

Command

- clear คำสั่งที่ใช้ในการเคลียร์หน้าจอ

```
Cmdr
chalersak@Takeit-Youth:~$ ls
111.txt 222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack train-jenkins wordpress-old
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ clear|
```

Linux Basic Command

Command

- sudo คำสั่งเรียกใช้งาน superuser

λ Cmdr

```
chalermsak@Takeit-Youth:~/test$ sudo touch 111.txt
[sudo] password for chalermsak:
chalermsak@Takeit-Youth:~/test$ ll
total 8
drwxr-xr-x  2 chalermsak chalermsak 4096 Apr  4 22:56 ./
drwxr-xr-x 19 chalermsak chalermsak 4096 Apr  4 22:56 ../
-rw-r--r--  1 root       root         0 Apr  4 22:56 111.txt
chalermsak@Takeit-Youth:~/test$
```

Linux Basic Command

Command

- vi, nano คำสั่งในการ create และ edit ไฟล์

Cmdr

```
chalermsak@Takeit-Youth:~/test$ vi 333.txt
chalermsak@Takeit-Youth:~/test$ ll
total 16
drwxr-xr-x  2 chalermsak chalermsak 4096 Apr  4 23:05 ./
drwxr-xr-x 19 chalermsak chalermsak 4096 Apr  4 23:05 ../
-rw-r--r--  1 root      root          0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalermsak chalermsak    0 Apr  4 22:58 222.txt
-rw-r--r--  1 chalermsak chalermsak    4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalermsak chalermsak    4 Apr  4 23:04 444.txt
chalermsak@Takeit-Youth:~/test$ nano 444.txt
chalermsak@Takeit-Youth:~/test$ ll
total 16
drwxr-xr-x  2 chalermsak chalermsak 4096 Apr  4 23:05 ./
drwxr-xr-x 19 chalermsak chalermsak 4096 Apr  4 23:05 ../
-rw-r--r--  1 root      root          0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalermsak chalermsak    0 Apr  4 22:58 222.txt
-rw-r--r--  1 chalermsak chalermsak    4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalermsak chalermsak    4 Apr  4 23:04 444.txt
chalermsak@Takeit-Youth:~/test$
```

Linux Basic Command

Command

- cat คำสั่งในการอ่านไฟล์

```
chalermsak@Takeit-Youth:~$ ls
111.txt  222.txt  scheduler  test1.txt
chalermsak@Takeit-Youth:~$ cat test1.txt
111111
222222
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ |
```

Linux Basic Command

Command

- mkdir ใช้สำหรับสร้าง folder

λ Cmder

```
chalersak@Takeit-Youth:~/test$ mkdir test1
chalersak@Takeit-Youth:~/test$ ll
total 20
drwxr-xr-x  3 chalersak chalersak 4096 Apr  4 23:07 ./
drwxr-xr-x 19 chalersak chalersak 4096 Apr  4 23:05 ../
-rw-r--r--  1 root      root          0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalersak chalersak      0 Apr  4 22:58 222.txt
-rw-r--r--  1 chalersak chalersak      4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalersak chalersak      4 Apr  4 23:04 444.txt
drwxr-xr-x  2 chalersak chalersak 4096 Apr  4 23:07 test1/
chalersak@Takeit-Youth:~/test$
```

Linux Basic Command

Command

- cd คำสั่งใช้ในการเข้าไปใน folder
- cd .. คำสั่งที่ใช้ใน back ออกจาก folder 1 step

```
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ ls  
111.txt 222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack test.txt test1 train-jenkins wordpress-old  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ cd test1/  
chalersak@Takeit-Youth:~/test1$  
chalersak@Takeit-Youth:~/test1$  
chalersak@Takeit-Youth:~/test1$ ls  
chalersak@Takeit-Youth:~/test1$  
chalersak@Takeit-Youth:~/test1$ cd ..  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ ls  
111.txt 222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack test.txt test1 train-jenkins wordpress-old  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ |
```

Linux Basic Command

Command

- rmdir คำสั่งที่ใช้ในการลบ folder

```
chalersak@Takeit-Youth:~$ ls
111.txt 222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack test.txt test1 train-jenkins wordpress-old
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ rmdir test1/
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ ls
111.txt 222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack test.txt train-jenkins wordpress-old
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$
```


Linux Basic Command

Command

- rm คำสั่งที่ใช้ลบไฟล์
- rm -r คำสั่งที่ใช้ในการลบไฟล์ และ folder

```
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ ls  
111.txt 222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack test1 train-jenkins wordpress-old  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ rm 111.txt  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ ls  
222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack test1 train-jenkins wordpress-old  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ rm test1/  
rm: cannot remove 'test1/': Is a directory  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ rm -r test1/  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ ls  
222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack train-jenkins wordpress-old  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$
```

Linux Basic Command

Command

- pwd คำสั่งที่ใช้ในการ check ที่อยู่ปัจจุบันแบบเต็ม check path directory

```
chalermsak@Takeit-Youth:~$ ls
222.txt database edoc edoc-oracle jenkins myproject nginx script st-old stack train-jenkins wordpress-old
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ pwd
/home/chalermsak
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$
```

Linux Basic Command

Command

- cp คำสั่งในการคัดลอกไฟล์

```
Cmder
chalersak@Takeit-Youth:~/test$ ll
total 20
drwxr-xr-x  3 chalersak chalersak 4096 Apr  4 23:07 ./
drwxr-xr-x 19 chalersak chalersak 4096 Apr  4 23:05 ../
-rw-r--r--  1 root      root          0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalersak chalersak      0 Apr  4 22:58 222.txt
-rw-r--r--  1 chalersak chalersak      4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalersak chalersak      4 Apr  4 23:04 444.txt
drwxr-xr-x  2 chalersak chalersak 4096 Apr  4 23:07 test1/
chalersak@Takeit-Youth:~/test$ cp 111.txt test1/
chalersak@Takeit-Youth:~/test$ ll test1/
total 8
drwxr-xr-x 2 chalersak chalersak 4096 Apr  4 23:09 ./
drwxr-xr-x 3 chalersak chalersak 4096 Apr  4 23:07 ../
-rw-r--r-- 1 chalersak chalersak    0 Apr  4 23:09 111.txt
chalersak@Takeit-Youth:~/test$
```

Linux Basic Command

Command

- mv คำสั่งในการย้ายไฟล์

```
Cmdr
chalersak@Takeit-Youth:~/test$ ll
total 20
drwxr-xr-x  3 chalersak chalersak 4096 Apr  4 23:11 ./
drwxr-xr-x 19 chalersak chalersak 4096 Apr  4 23:05 ../
-rw-r--r--  1 root      root      0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalersak chalersak  0 Apr  4 22:58 222.txt
-rw-r--r--  1 chalersak chalersak  4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalersak chalersak  4 Apr  4 23:04 444.txt
drwxr-xr-x  2 chalersak chalersak 4096 Apr  4 23:11 test1/
chalersak@Takeit-Youth:~/test$ mv 222.txt test1/
chalersak@Takeit-Youth:~/test$ ll
total 20
drwxr-xr-x  3 chalersak chalersak 4096 Apr  4 23:11 ./
drwxr-xr-x 19 chalersak chalersak 4096 Apr  4 23:05 ../
-rw-r--r--  1 root      root      0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalersak chalersak  4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalersak chalersak  4 Apr  4 23:04 444.txt
drwxr-xr-x  2 chalersak chalersak 4096 Apr  4 23:11 test1/
chalersak@Takeit-Youth:~/test$ ll test1/
total 8
drwxr-xr-x 2 chalersak chalersak 4096 Apr  4 23:11 ./
drwxr-xr-x 3 chalersak chalersak 4096 Apr  4 23:11 ../
-rw-r--r-- 1 chalersak chalersak  0 Apr  4 23:09 111.txt
-rw-r--r-- 1 chalersak chalersak  0 Apr  4 22:58 222.txt
chalersak@Takeit-Youth:~/test$
```

Linux Basic Command

Command

- chmod คำสั่งการจัดการสิทธิ์ในการเรียกใช้ ไฟล์ และ folder

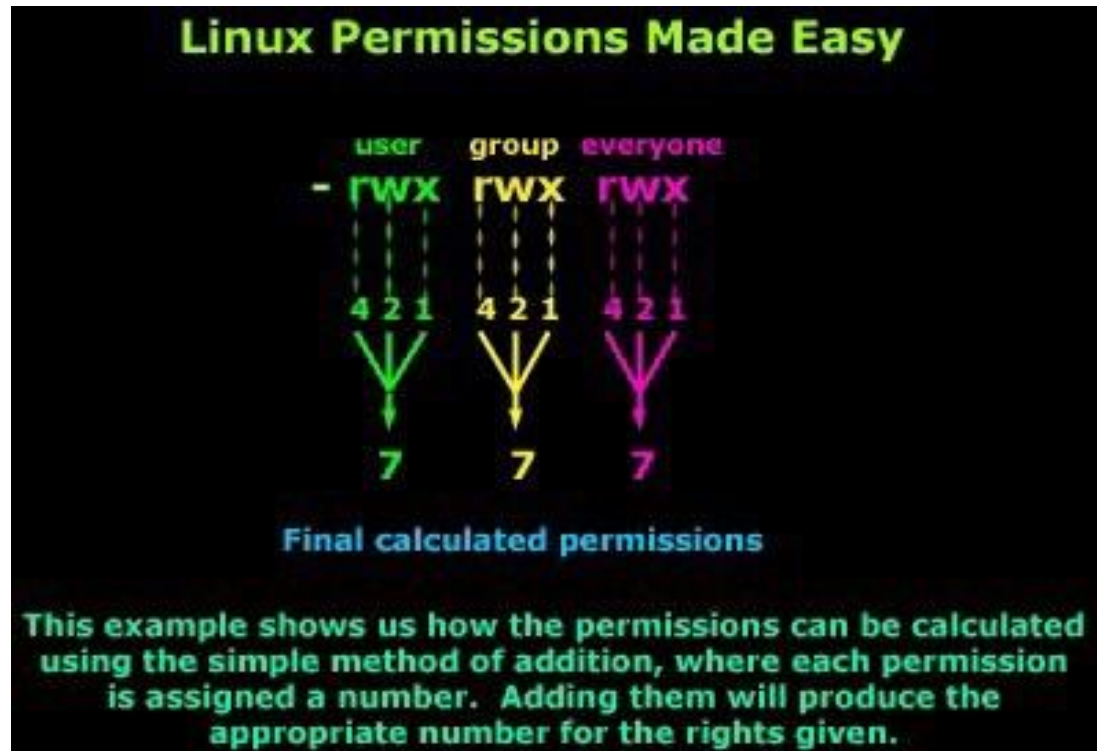
Cmdr

```
chalersak@Takeit-Youth:~/test$ ll
total 20
drwxr-xr-x  3 chalersak chalersak 4096 Apr  4 23:11 ./
drwxr-xr-x 20 chalersak chalersak 4096 Apr  4 23:14 ../
-rw-r--r--  1 root      root          0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalersak chalersak      4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalersak chalersak      4 Apr  4 23:04 444.txt
drwxr-xr-x  2 chalersak chalersak 4096 Apr  4 23:11 test1/
chalersak@Takeit-Youth:~/test$ sudo chmod 777 test1/
chalersak@Takeit-Youth:~/test$ ll
total 20
drwxr-xr-x  3 chalersak chalersak 4096 Apr  4 23:11 ./
drwxr-xr-x 20 chalersak chalersak 4096 Apr  4 23:14 ../
-rw-r--r--  1 root      root          0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalersak chalersak      4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalersak chalersak      4 Apr  4 23:04 444.txt
drwxrwxrwx  2 chalersak chalersak 4096 Apr  4 23:11 test1/
chalersak@Takeit-Youth:~/test$
```

Linux Basic Command

Command

- Linux Permissions



Linux Basic Command

Command

- chown คำสั่งที่ใช้ในการให้สิทธิ์ในการให้ความเป็นเจ้าของไฟล์ และ folder

```
Cmder
chalersak@Takeit-Youth:~/test$ ll
total 20
drwxr-xr-x  3 chalerm sak chalerm sak 4096 Apr  4 23:11 ./
drwxr-xr-x 20 chalerm sak chalerm sak 4096 Apr  4 23:14 ../
-rw-r--r--  1 root      root          0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalerm sak chalerm sak  4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalerm sak chalerm sak  4 Apr  4 23:04 444.txt
drwxrwxrwx  2 chalerm sak chalerm sak 4096 Apr  4 23:11 test1/
chalersak@Takeit-Youth:~/test$ sudo chown chalerm sak:chalerm sak 111.txt
chalersak@Takeit-Youth:~/test$ ll
total 20
drwxr-xr-x  3 chalerm sak chalerm sak 4096 Apr  4 23:11 ./
drwxr-xr-x 20 chalerm sak chalerm sak 4096 Apr  4 23:14 ../
-rw-r--r--  1 chalerm sak chalerm sak  0 Apr  4 22:56 111.txt
-rw-r--r--  1 chalerm sak chalerm sak  4 Apr  4 23:02 333.txt
-rw-r--r--  1 chalerm sak chalerm sak  4 Apr  4 23:04 444.txt
drwxrwxrwx  2 chalerm sak chalerm sak 4096 Apr  4 23:11 test1/
chalersak@Takeit-Youth:~/test$ |
```

Linux Basic Command

Command

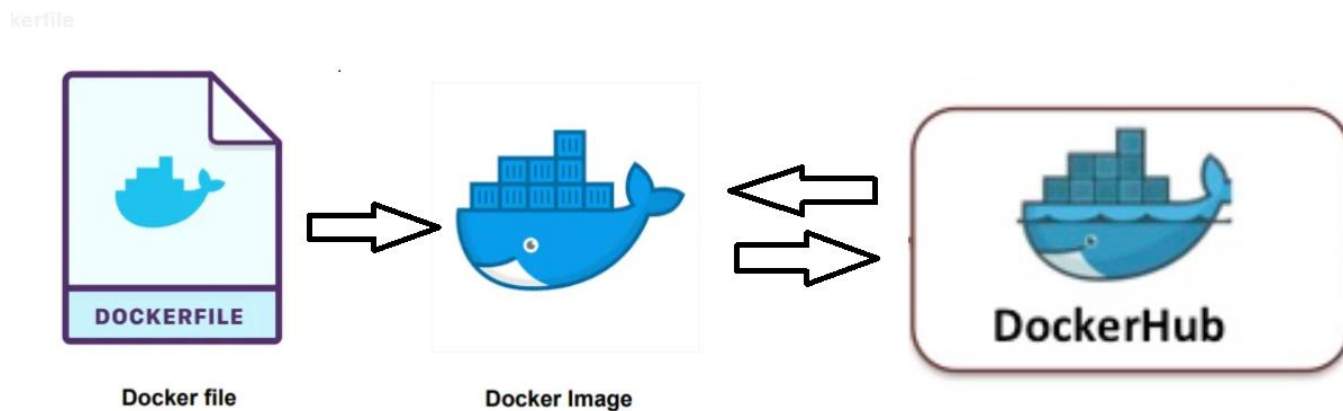
- `exit` คำสั่งที่ใช้ในออกจาก Container หรือเราสามารถออกจาก `cmdr` ของเราก็ได้เช่นกัน

```
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ exit  
logout  
  
C:\tools\Cmder  
λ  
C:\tools\Cmder  
λ
```


Docker images

Docker images คืออะไร

Docker Image คือ สิ่งที่เราสร้างขึ้นมาเพื่อเป็นพิมพ์เขียวให้กับ Docker ในการนำไปสร้าง Docker Containers เพราะฉะนั้นแล้วในการที่เราจะสร้าง Docker Containers ขึ้นมา เราจะต้องทำการสร้าง Docker Image ขึ้นมาก่อน ซึ่ง Docker ก็มี hub.docker.com ที่เป็นแหล่งรวม Docker Image ที่เราสามารถเอา Image ของเรามาแชร์กับคนอื่นได้ รวมไปถึงเราสามารถใช้อะไรของคนอื่นก็ได้เช่นกัน



Docker images

Docker images pull

- `docker pull hello-world` เพื่อ pull images มาใช้งาน
- `docker images` เพื่อตรวจสอบว่ามีการ pull images มาแล้ว.

```
λ Cmdr
chalermsak@Takeit-Youth:~$ docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
2db29710123e: Pull complete
Digest: sha256:fffb13da98453e0f04d33a6eee5bb8e46ee50d08ebe17735fc0779d0349e889e9
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
chalermsak@Takeit-Youth:~$ docker images
REPOSITORY      TAG          IMAGE ID      CREATED        SIZE
hello-world     latest      feb5d9fea6a5  18 months ago  13.3kB
chalermsak@Takeit-Youth:~$
```

Docker images

Docker images command

- `docker images` เพื่อตรวจสอบว่ามี images ตัวไหนอยู่บ้าง
- `docker image ls` เพื่อตรวจสอบว่ามี images ตัวไหนอยู่บ้าง
- `docker images --no-trunc` เพื่อตรวจสอบว่ามี images ตัวไหนอยู่บ้าง
- `docker rmi repository:tag` เพื่อลบ images ที่มีอยู่

```
Cmder
chalersak@Takeit-Youth:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    feb5d9fea6a5   18 months ago  13.3kB
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    feb5d9fea6a5   18 months ago  13.3kB
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker images --no-trunc
REPOSITORY    TAG       IMAGE ID                                     CREATED        SIZE
hello-world   latest    sha256:feb5d9fea6a5e9606aa995e879d862b825965ba48de054caab5ef356dc6b3412  18 months ago  13.3kB
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker rmi hello-world:latest
Untagged: hello-world:latest
Untagged: hello-world@sha256:ffb13da98453e0f04d33a6eee5bb8e46ee50d08ebe17735fc0779d0349e889e9
Deleted: sha256:feb5d9fea6a5e9606aa995e879d862b825965ba48de054caab5ef356dc6b3412
Deleted: sha256:e07ee1baac5fae6a26f30cabfe54a36d3402f96afda318fe0a96cec4ca393359
chalersak@Takeit-Youth:~$
```

Docker images ?

Docker volume Command

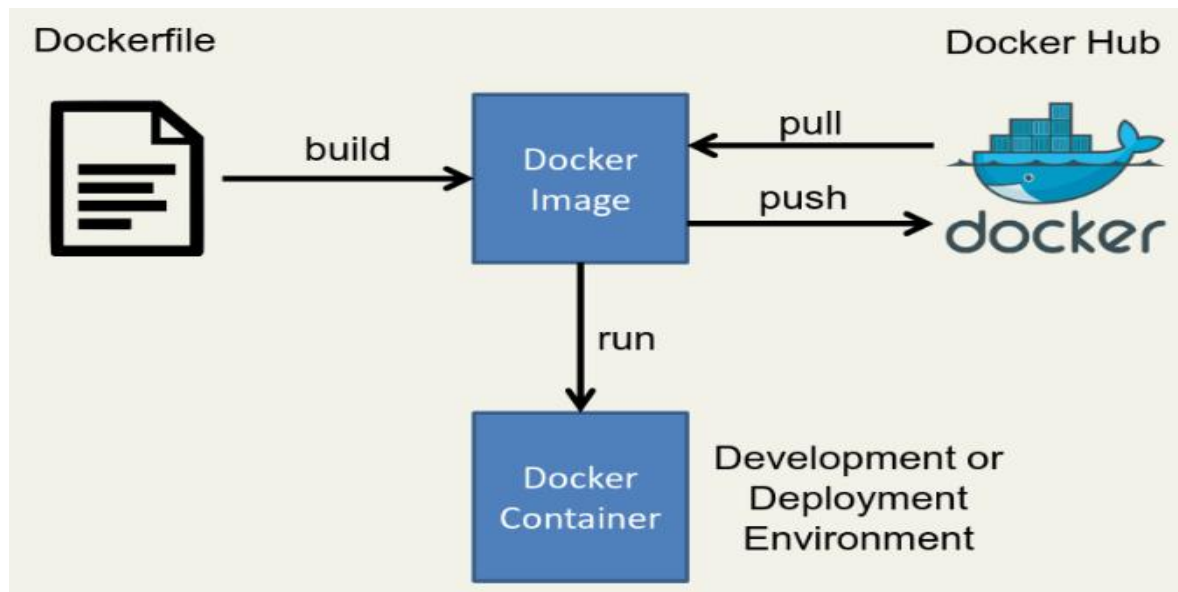
- `docker image inspect <image>` คำสั่งที่ใช้ตรวจสอบรายละเอียดของ images

```
chalermsak@Takeit-Youth:~$ docker image inspect hello-world
[
  {
    "Id": "sha256:d2c94e258dcb3c5ac2798d32e1249e42ef01cba4841c2234249495f87264ac5a",
    "RepoTags": [
      "hello-world:latest"
    ],
    "RepoDigests": [
      "hello-world@sha256:1408fec50309afee38f3535383f5b09419e6dc0925bc69891e79d84cc4cdcec6"
    ],
    "Parent": "",
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2023-05-02T16:49:27Z",
    "ContainerConfig": {
      "Hostname": "",
      "Domainname": "",
      "User": "",
      "AttachStdin": false,
      "AttachStdout": false,
      "AttachStderr": false,
      "Tty": false,
      "OpenStdin": false,
      "StdinOnce": false,
      "Env": null,
      "Cmd": null,
      "Image": "",
      "Volumes": null,
      "WorkingDir": "",
      "Entrypoint": null,
      "OnBuild": null,
      "Labels": null
    },
    "DockerVersion": "",
    "Author": "",
    "Config": {
      "Hostname": "",
      "Domainname": "",
      "User": "",
      "AttachStdin": false,
      "AttachStdout": false,
```

Docker container

Docker container คืออะไร

- Docker Container เป็นที่บรรจุรวมของแอปพลิเคชัน สภาพแวดล้อมที่จำเป็นและ องค์ประกอบต่างๆ ที่จำเป็นต่อการทำงาน ซึ่งสามารถสร้างจาก Docker Image ผ่านการกำหนด โครงสร้างของ Docker File หรือ สามารถ pull images จาก hub.docker.com และทำการ run ด้วยคำสั่ง docker run ได้เช่นกัน



Docker container

Docker command

- `docker run hello-world` will run images hello-world

Cmdr

```
chalermsak@Takeit-Youth:~$ docker run hello-world
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

Docker container

Docker command

- docker run กับ docker pull แล้ว run start จริงๆแล้วเหมือนกัน แต่จะทำกันคนละ step

Docker run hello-world

Cmder

```
chalermsak@Takeit-Youth:~$ docker run hello-world
```

```
Hello from Docker!  
This message shows that your installation appears to be working correctly.
```

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
\$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:
<https://docs.docker.com/get-started/>

Docker pull แล้ว run start

```
chalermsak@Takeit-Youth:~$ docker pull hello-world  
Using default tag: latest  
latest: Pulling from library/hello-world  
Digest: sha256:53cc4d415d839c98be39331c948609b659ed725170ad2ca8eb36951288f81b75  
Status: Image is up to date for hello-world:latest  
docker.io/library/hello-world:latest
```

```
chalermsak@Takeit-Youth:~$ docker images  
REPOSITORY TAG IMAGE ID CREATED SIZE  
hello-world latest d2c94e258dc15 months ago 13.3kB
```

```
chalermsak@Takeit-Youth:~$ docker run hello-world
```

```
Hello from Docker!  
This message shows that your installation appears to be working correctly.
```

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
\$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:
<https://docs.docker.com/get-started/>

```
chalermsak@Takeit-Youth:~$
```

Docker container

Docker command

- `docker ps` คือคำสั่งในการ check Process เพื่อตรวจสอบว่ามี container ตัวไหนถูก Start อยู่บ้าง
- `docker container ls` เพื่อตรวจสอบว่ามี container ตัวไหนถูก Start อยู่

λ Cmder

```
chalersak@Takeit-Youth:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker container ls
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ |
```


Docker container

Docker command

- `docker ps -a` คือคำสั่งในการ check Process เพื่อตรวจสอบว่ามี container ทั้งหมด ทั้งที่ถูก start และไม่ start อยู่ก็ตัว
- `docker container ls -a` เพื่อตรวจสอบว่ามี container ทั้งหมด ทั้งที่ถูก start และ ไม่ start อยู่ก็ตัว

Cmder

```
chalermsak@Takeit-Youth:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
0aa121014bd7   hello-world    "/hello"                 3 minutes ago Exited (0) 3 minutes ago           bold_goodall
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker container ls -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
0aa121014bd7   hello-world    "/hello"                 3 minutes ago Exited (0) 3 minutes ago           bold_goodall
chalermsak@Takeit-Youth:~$
```

Docker container

Docker command

Parameter ที่ใช้ใน Container

- `--rm` คือ การ remove container เมื่อมีการ exit ออกจาก container
- `-d` detached คือ เมื่อ run container ขึ้นมามันจะคืน cursor ให้เราและ container จะรันแยกจาก เทอร์มินัลที่กำลังใช้งานอยู่
- `--name` คือ การตั้งชื่อ Container_name
- `-p` คือ การกำหนด port หรือ การ expose port
- `-it` คือ interactive กับ tty การเปิดการเชื่อมต่อกับ Container สามารถเข้าไปใช้งานใน Container ได้
- `logs` คือ การ check logs การทำงานของ Container
- `-e` คือ การสร้าง environment เพื่อกำหนดค่าตัวแปรที่ใช้สำหรับ Container

Docker container

Docker command

- `docker run --rm -it busybox` คือ การ run container แล้ว shell เข้าไปใน container และเมื่อออกจาก Container แล้ว

Container ก็จะถูก rm ทิ้งด้วย

```
chalerssak@Takeit-Youth:~$ docker run -it --rm busybox sh
/ #
/ #
/ #
/ # ls
bin      dev      etc      home     lib      lib64    proc     root     sys      tmp      usr      var
/ #
/ #
/ # cat /etc/hosts
127.0.0.1      localhost
::1           localhost ip6-localhost ip6-loopback
fe00::0       ip6-localnet
ff00::0       ip6-mcastprefix
ff02::1       ip6-allnodes
ff02::2       ip6-allrouters
10.10.0.2      73f816ae5d51
/ #
/ #
/ # exit
```

```
chalerssak@Takeit-Youth:~$
chalerssak@Takeit-Youth:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
chalerssak@Takeit-Youth:~$
chalerssak@Takeit-Youth:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
chalerssak@Takeit-Youth:~$
chalerssak@Takeit-Youth:~$
chalerssak@Takeit-Youth:~$ |
```

Docker container

Docker command

- `docker run -d --name nginx_web -p 80:80 -e TZ=Asia/Bangkok nginx`
- เมื่อใช้คำสั่ง `-e TZ=Asia/Bangkok` คือการกำหนด date time ที่อยู่ใน container

Run โดยไม่ได้กำหนด date time ใน container

```
chalersak@Takeit-Youth:~$ docker run -d nginx
0aaa7ca14c7de921745e6308d0de84824cdc14659b53384cc5e8c7f7c3466598
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ date
Wed Aug 14 17:31:13 +07 2024
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker exec -it blissful_gagarin bash
root@0aaa7ca14c7d:/#
root@0aaa7ca14c7d:/# date
Wed Aug 14 10:31:21 UTC 2024
root@0aaa7ca14c7d:/#
root@0aaa7ca14c7d:/#
root@0aaa7ca14c7d:/#
```

Run โดยกำหนด date time ใน container

```
chalersak@Takeit-Youth:~$ docker run -d -e TZ=Asia/Bangkok nginx
090371e6a86c489d302d1ea4c1ebd5cb6cb291c84f1de67a21f5144fc155daf2
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ date
Wed Aug 14 17:33:36 +07 2024
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker exec -it relaxed_robinson bash
root@090371e6a86c:/#
root@090371e6a86c:/#
root@090371e6a86c:/# date
Wed Aug 14 17:33:43 +07 2024
root@090371e6a86c:/#
root@090371e6a86c:/#
root@090371e6a86c:/#
```

Docker container

Docker command

- `docker stop <CONTAINER ID หรือ NAMES>` เพื่อ Stop container ที่ถูก Start อยู่

Cmdr

```
chalermsak@Takeit-Youth:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
c6d4caceb256   nginx    "/docker-entrypoint...." 10 minutes ago Up 2 seconds  80/tcp       kind_jepsen
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker stop c6d
c6d
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
chalermsak@Takeit-Youth:~$
```

Docker container

Docker command

- `docker start <CONTAINER ID หรือ NAMES>` เพื่อ start container ที่ถูก stop ไปแล้ว

Cmdr

```
chalersak@Takeit-Youth:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS      NAMES
c6d4caceb256   nginx    "/docker-entrypoint...." 12 minutes ago Exited (0) 2 minutes ago      kind_jepsen
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker start c6d
c6d
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS      NAMES
c6d4caceb256   nginx    "/docker-entrypoint...." 12 minutes ago Up 6 seconds   80/tcp      kind_jepsen
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ |
```

Docker container

Docker command

`docker logs [options] [container_name_or_id]` ใช้เพื่อดู **log** ที่ถูกสร้างขึ้น โดยคอนเทนเนอร์ ซึ่งเป็นบันทึกการทำงานของคอนเทนเนอร์นั้นๆ

Docker container

Docker command

- `docker container inspect < CONTAINER ID หรือ NAMES >` คำสั่งใช้ตรวจสอบรายละเอียดของ Container

```
chalermsak@Takeit-Youth:~/nginx$ docker container inspect nginx-web-1
[
  {
    "Id": "79fa6df3a712179b37c33ac7dc2b45c53866da948f9974dcf78304bd2e4523b0",
    "Created": "2024-08-07T08:08:21.564351863Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 681,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2024-08-07T08:08:22.290057122Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:a72860cb95fd59e9c696c66441c64f18e66915fa26b249911e83c3854477ed9a",
    "ResolvConfPath": "/var/lib/docker/containers/79fa6df3a712179b37c33ac7dc2b45c53866da948f9974dcf78304bd2e4523b0/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/79fa6df3a712179b37c33ac7dc2b45c53866da948f9974dcf78304bd2e4523b0/hostname",
    "HostsPath": "/var/lib/docker/containers/79fa6df3a712179b37c33ac7dc2b45c53866da948f9974dcf78304bd2e4523b0/hosts",
    "LogPath": "/var/lib/docker/containers/79fa6df3a712179b37c33ac7dc2b45c53866da948f9974dcf78304bd2e4523b0/79fa6df3a712179b37c33ac7dc2b45c53866da948f9974dcf78304bd2e4523b0-json.log",
    "Name": "/nginx-web-1",
    "RestartCount": 0,
    "Driver": "overlay2",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": [
```


Docker container

Docker command

- `docker rm <CONTAINER ID หรือ NAMES>` เพื่อลบ container แต่ container นั้นต้องถูก stop ไว้ก่อนจึงจะใช้งานได้

Cmdr

```
chalersak@Takeit-Youth:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
c6d4caceb256   nginx    "/docker-entrypoint..." 15 minutes ago Up 3 minutes   80/tcp       kind_jepsen
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker stop c6d
c6d
chalersak@Takeit-Youth:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
c6d4caceb256   nginx    "/docker-entrypoint..." 15 minutes ago Exited (0) 7 seconds ago          kind_jepsen
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker rm c6d
c6d
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
chalersak@Takeit-Youth:~$ |
```

Docker container

Docker command

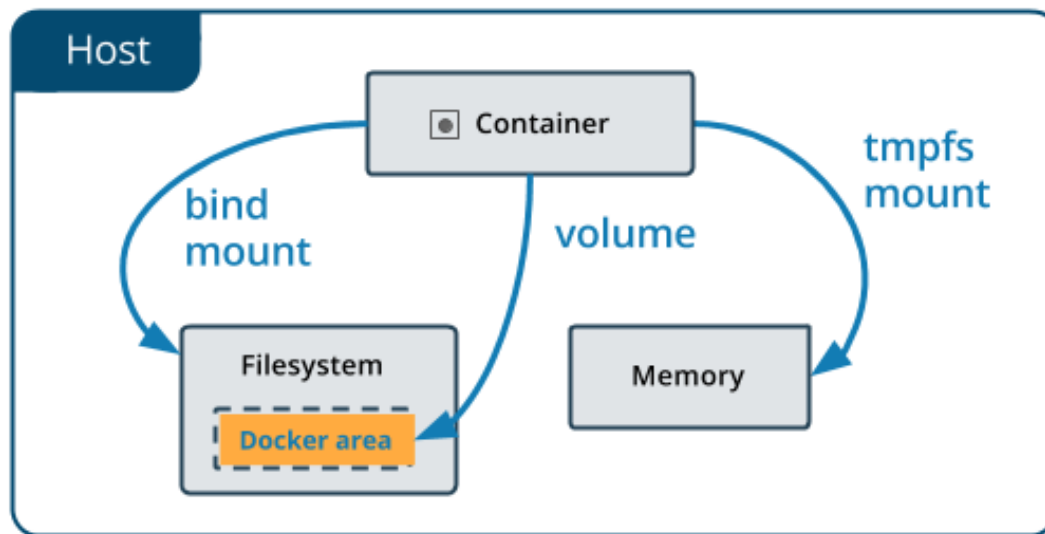
- แบบฝึกหัด run container mariadb database โดยมีการใช้
 - โดยใช้ชื่อเป็น mariadb-container
 - Port เป็น 3307:3306
 - ใส่ -d เพื่อ จะคืน cursor กลับมาด้วย
 - ใช้ environment เป็น -e MARIADB_ROOT_PASSWORD=123456789 -e MARIADB_DATABASE=maria_db
 - Image เป็น mariadb:latest (เปิด hub.docker.com)

Docker Volume ?

Docker volume คืออะไร

docker volume คือ การแชร์ข้อมูลระหว่าง docker host กับ container ภายนอกและภายใน โดยภายนอกก็คือ directory path ที่ทำการ run docker และภายในคือ directory path ที่อยู่ใน container การใช้ Volume นั้น สามารถใช้งานได้ 2 แบบด้วยกัน

- -v
- -mount



Docker Volume ?

Docker volume Command

- `docker volume create <volume_name>` คำสั่งที่ใช้ในการสร้าง volume

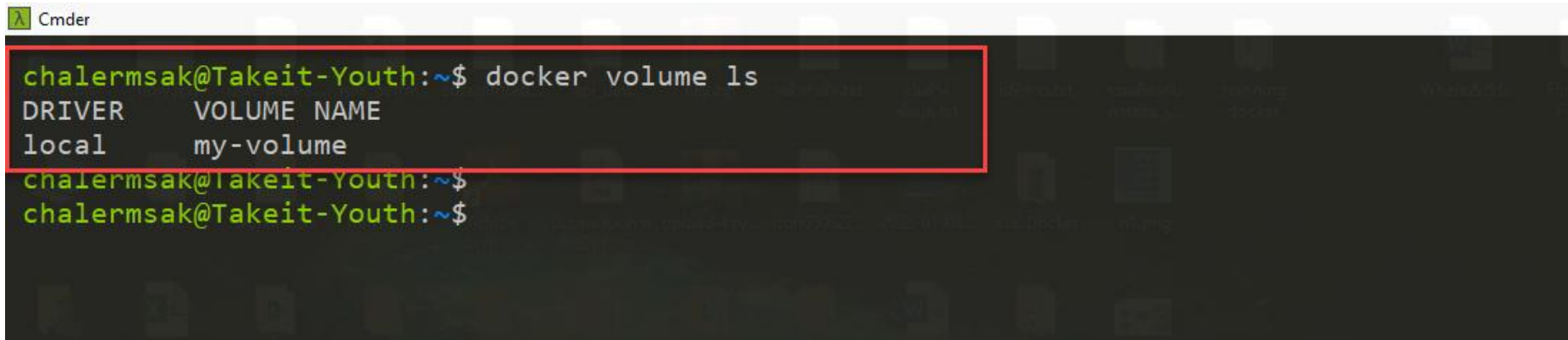
λ Cmdr

```
chalermsak@Takeit-Youth:~$ docker volume create my-volume  
my-volume  
chalermsak@Takeit-Youth:~$  
chalermsak@Takeit-Youth:~$
```

Docker Volume ?

Docker volume Command

- `docker volume ls` คำสั่งที่ใช้ในการตรวจสอบ ว่ามี volume ตัวไหนอยู่บ้าง

A screenshot of a Windows Command Prompt window titled 'Cmder'. The prompt shows the user 'chalersak@Takeit-Youth' at the shell '~'. The command 'docker volume ls' has been entered and executed. The output is a table with two columns: 'DRIVER' and 'VOLUME NAME'. The first row shows 'local' as the driver and 'my-volume' as the volume name. The prompt then shows the user entering another 'docker volume ls' command, which returns the same output. The first command and its output are highlighted with a red rectangular box.

```
chalersak@Takeit-Youth:~$ docker volume ls
DRIVER      VOLUME NAME
local       my-volume
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker volume ls
DRIVER      VOLUME NAME
local       my-volume
```

Docker Volume ?

Docker volume Command

- `docker volume inspect <volume_name>` คำสั่งที่ใช้ตรวจสอบรายละเอียดของ volume

```
Cmdr
chalermsak@Takeit-Youth:~$ docker volume inspect my-volume
[
  {
    "CreatedAt": "2023-04-06T14:33:32Z",
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/my-volume/_data",
    "Name": "my-volume",
    "Options": {},
    "Scope": "local"
  }
]
chalermsak@Takeit-Youth:~$ |
```

Docker Volume ?

Docker volume

-v คือ เป็นการระบุแบบใช้ : (Colon) เป็นตัวคั่นระหว่างภายนอก Container และภายใน Container

ตัวอย่าง

- 1 `docker run --name nginx -v my-volume-nginx:/var/log/nginx -d nginx`
- 2 `docker run --name nginx -v $(pwd)/nginx:/var/log/nginx -d nginx`

1. จะเป็นการ mount volume โดยการ create volume และนำไป mount กับ /var/log/nginx ที่อยู่ใน container
 - `docker run --name nginx5 -v volume-nginx:/var/log/nginx -d nginx`
2. จะเป็นการ mount volume โดยการสร้าง folder และนำไป mount กับ /var/log/nginx ที่อยู่ใน container
 - `docker run --name nginx6 -v $(pwd)/nginx:/var/log/nginx -d nginx`

Docker Volume ?

Docker volume

--mount คือ เป็นการระบุแบบใช้ <key>=<value> แล้วกันด้วย ,

- `docker run -d --name nginx --mount source=my-volume,target=/var/log/nginx nginx`
1. จะเป็นการ mount volume โดยการระบุ source คือ my-volume ที่ทำการสร้างขึ้นมา และนำไป mount กับ target คือ /var/log/nginx ที่อยู่ใน container
 - `docker run -d --name nginx7 --mount source=volume-nginx1,target=/var/log/nginx nginx`

Docker Volume ?

Create MySQL mount volumes

- `docker run --name mariadb-container -v $(pwd)/maria_database:/var/lib/mysql -e MARIADB_ROOT_PASSWORD=123456789 -e MARIADB_DATABASE=maria_db -p 3307:3306 -d mariadb`

```
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ mkdir mysql  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$  
chalersak@Takeit-Youth:~$ cd mysql/  
chalersak@Takeit-Youth:~/mysql$  
chalersak@Takeit-Youth:~/mysql$  
chalersak@Takeit-Youth:~/mysql$ docker run --name mysqldb -v $(pwd)/database:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=1234 -e MYSQL_DATABASE=sqldb -p 3306:3306  
-d mysql  
Unable to find image 'mysql:latest' locally  
latest: Pulling from library/mysql  
d9a40b27c30f: Pull complete  
fe4b01031aab: Pull complete  
aa72c34c4347: Pull complete  
473ade985fa2: Pull complete  
cc168a9482de: Pull complete  
3ca3786815dd: Pull complete  
3e3fac98ea83: Pull complete  
10e5505c3ae4: Pull complete  
a79ade39aab9: Pull complete  
ae34d51c6da2: Pull complete  
Digest: sha256:d8df069848906979fd7511db00dc22efeb0a33a990d87c3c6d3fcdafd6fc6123  
Status: Downloaded newer image for mysql:latest  
ad242dc6aada6ff26a38e05a735016c2e5d26d573ddc3248ac1f5e282cd87ebd  
chalersak@Takeit-Youth:~/mysql$  
chalersak@Takeit-Youth:~/mysql$ docker ps  
CONTAINER ID   IMAGE     COMMAND                  CREATED       STATUS       PORTS                               NAMES  
ad242dc6aada   mysql    "docker-entrypoint.s..." 8 seconds ago Up 6 seconds 0.0.0.0:3306->3306/tcp, 33060/tcp  mysqldb  
chalersak@Takeit-Youth:~/mysql$  
chalersak@Takeit-Youth:~/mysql$
```

Docker Volume ?

Docker volume Command

- `docker volume rm <volume_name>` คำสั่งที่ใช้ remove volume

Cmdr

```
chalermsak@Takeit-Youth:~$ docker volume ls
```

DRIVER	VOLUME NAME
local	my-volume

```
chalermsak@Takeit-Youth:~$
```

```
chalermsak@Takeit-Youth:~$
```

```
chalermsak@Takeit-Youth:~$ docker volume rm my-volume  
my-volume
```

```
chalermsak@Takeit-Youth:~$
```

```
chalermsak@Takeit-Youth:~$ docker volume ls
```

DRIVER	VOLUME NAME
--------	-------------

```
chalermsak@Takeit-Youth:~$
```

```
chalermsak@Takeit-Youth:~$ |
```

Docker Volume ?

Docker volume Command

- `docker cp <file> <container_name>:<path>`

```
chalermsak@Takeit-Youth:~$ ls
111.txt  222.txt  database  edoc  edoc-oracle  jenkins  myproject  nginx  script  st-old  stack  train-jenkins  wordpress-old
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
892066a4a6ce   nginx     "/docker-entrypoint..." 3 minutes ago  Up 3 minutes  80/tcp         admiring_jemison
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker cp 111.txt admiring_jemison:/
Successfully copied 1.54kB to admiring_jemison:/
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker exec -it admiring_jemison bash
root@892066a4a6ce:/#
root@892066a4a6ce:/#
root@892066a4a6ce:/# ls
111.txt  bin  boot  dev  docker-entrypoint.d  docker-entrypoint.sh  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@892066a4a6ce:/#
root@892066a4a6ce:/#
root@892066a4a6ce:/# |
```

Docker Volume ?

Docker volume

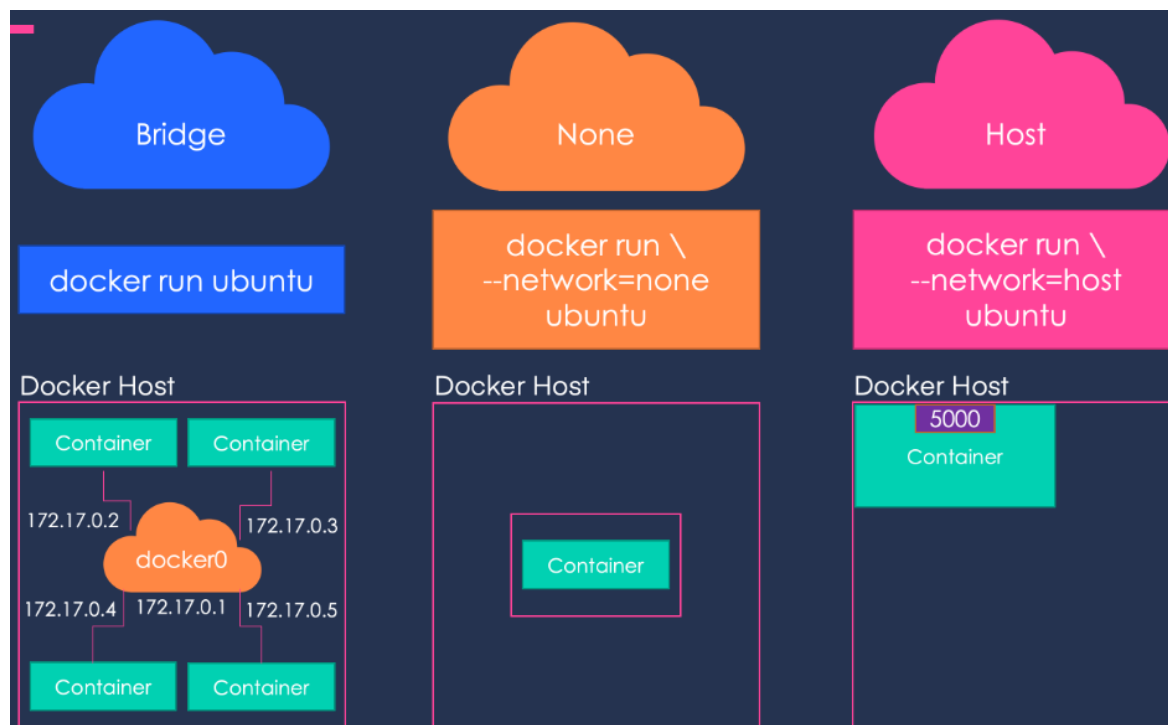
- ทดสอบ run container postgres database โดยมีการใช้
 - โดยใช้ชื่อเป็น postgres-container
 - Port เป็น 5433:5432
 - Mount volume โดยการสร้าง folder \$(pwd)/pg_database:/var/lib/postgresql/data
 - ใส่ -d เพื่อ จะคืน cursor กลับมาด้วย
 - ใช้ environment เป็น -e POSTGRES_PASSWORD=123456789 -e POSTGRES_USER=myuser -e POSTGRES_DB=pg_data
 - Image เป็น postgres:latest

Docker Network ?

Docker Network คืออะไร

เครือข่ายเสมือนใน Docker ทำงานโดยใช้ไดรเวอร์ หรือซอฟต์แวร์เป็นตัวขับเคลื่อนลักษณะการทำงาน โดยค่าเริ่มต้นมีอยู่ 3 ประเภท

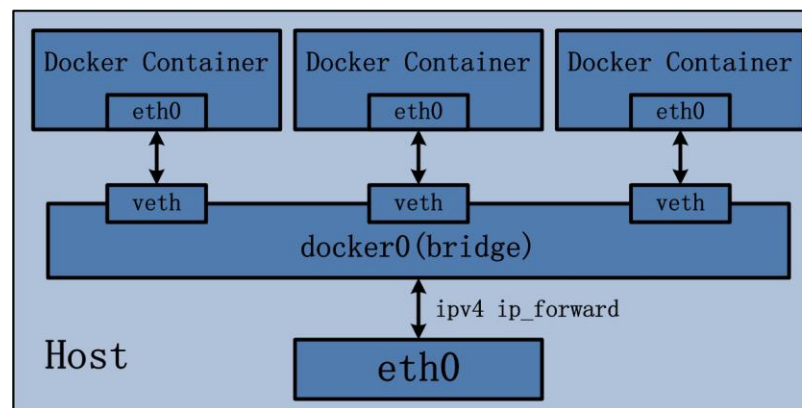
- bridge
- host
- none



Docker Network ?

Bridge Network

- ใช้งาน Internet จากภายใน container ได้
- Docker Engine จะสร้าง Subnet และ Gateway ของ network นี้ให้อัตโนมัติ
- เมื่อเรียกใช้งาน image แบบปกติ container ที่ถูกสร้างใหม่นั้นจะถูกกำหนดให้อยู่ใน network โดยอัตโนมัติ
- Container ที่อยู่ใน docker daemon เดียวกันคุยกันได้
- Network Bridge ของ Docker จะมีความคล้ายคลึงกับการทำ NAT ใน VMware หรือ VirtualBox ซึ่งใน Docker จะทำการสร้าง IP มาให้กับ Container มาให้ เพื่อ มาคุยกันผ่าน docker0 แล้ว การ์ด Networkinterface จะคุยกันผ่าน docker0 อีกที



Docker Network ?

Bridge Network

- ```
{ "default-address-pools": [
 {"base":"172.17.0.0/16","size":16}, <-- docker0
 {"base":"172.18.0.0/16","size":16}, <-- docker_gwbridge
 {"base":"172.19.0.0/16","size":16}, <-- ucp-bridge
 {"base":"172.20.0.0/16","size":16},
 {"base":"172.21.0.0/16","size":16},
 {"base":"172.22.0.0/16","size":16},
 {"base":"172.23.0.0/16","size":16},
 {"base":"172.24.0.0/16","size":16},
 {"base":"172.25.0.0/16","size":16},
 {"base":"172.26.0.0/16","size":16},
 {"base":"172.27.0.0/16","size":16},
 {"base":"172.28.0.0/16","size":16},
 {"base":"172.29.0.0/16","size":16},
 {"base":"172.30.0.0/16","size":16},
 {"base":"192.168.0.0/16","size":20}] }
```

# Docker Network ?

---

## Host Network

Host คือ network ที่ container ใช้ network interface ของ docker machine host ถ้าจัดระดับความปลอดภัยในการเข้าถึง container จัดได้ว่า host network นี้มีความปลอดภัยน้อยที่สุด

- container จะรันอยู่บน host network เลย ซึ่งสามารถ full access ได้จาก host interface ดังนั้นจะเรียก containers พวกนี้ว่า open containers
- ถ้า start container ใน port 5000 โดยใช้ network แบบ host port จะถูกใช้งานผ่านทาง host ได้เลยครับ ไม่ต้อง expose port ออกมา



# Docker Network ?

---

## None Network

none คือ container ในระบบปิด ที่ไม่สามารถออก network ภายนอกได้ ปกติมักสร้าง None network ไว้เชื่อมกับ custom network driver ดังนั้นใน container จะเห็นแค่ localhost interface

# Docker Network ?

## Docker network Command

- docker network ls เพื่อ check ว่ามี network อะไรบ้าง

λ Cmdr

```
chalersak@Takeit-Youth:~$ docker network ls
NETWORK ID NAME DRIVER SCOPE
c096d77a1999 bridge bridge local
4a8130cfd284 host host local
05da587d0283 none null local
chalersak@Takeit-Youth:~$
```

# Docker Network ?

## Docker network Command

- `docker network create <network name>` เพื่อสร้าง network ที่ต้องการจะใช้งาน

λ Cmdr

```
chalermsak@Takeit-Youth:~$ docker network create testnetwork
fec1dd152f0e6729f1bb4c65416a866bcc68eb03f93727122c5db47988191f8e
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker network ls
NETWORK ID NAME DRIVER SCOPE
c096d77a1999 bridge bridge local
4a8130cfd284 host host local
05da587d0283 none null local
fec1dd152f0e testnetwork bridge local
chalermsak@Takeit-Youth:~$
```

# Docker Network ?

## Docker network Command

- `docker network inspect <network name>`

เพื่อ check network ว่าใน network มี container อะไรที่  
อยู่ภายใต้ network นี้บ้าง และเอาไว้เพื่อ ดูข้อมูลต่างๆที่  
อยู่ใน network นั้นๆ

```
Cmdr
chalerm sak@Takeit-Youth:~$ docker network inspect testnetwork
[
 {
 "Name": "testnetwork",
 "Id": "fec1dd152f0e6729f1bb4c65416a866bcc68eb03f93727122c5db47988191f8e",
 "Created": "2023-04-06T11:16:17.3392819Z",
 "Scope": "local",
 "Driver": "bridge",
 "EnableIPv6": false,
 "IPAM": {
 "Driver": "default",
 "Options": {},
 "Config": [
 {
 "Subnet": "172.19.0.0/16",
 "Gateway": "172.19.0.1"
 }
]
 },
 "Internal": false,
 "Attachable": false,
 "Ingress": false,
 "ConfigFrom": {
 "Network": ""
 },
 "ConfigOnly": false,
 "Containers": {
 "56b84a34bd66bb71d60be4bf2caa23dc9a0378420ac25f34af964b0bd45b62d4": {
 "Name": "pensive_cray",
 "EndpointID": "a8cc64f83e2977189dd535794a7a4a3507dccccb3824e543aaa8acce907a6232",
 "MacAddress": "02:42:ac:13:00:02",
 "IPv4Address": "172.19.0.2/16",
 "IPv6Address": ""
 }
 },
 "Options": {},
 "Labels": {}
 }
]
chalerm sak@Takeit-Youth:~$
```

# Docker Network ?

## Docker network Command

- `docker network connect <network name> <container name>` เพื่อ connect ให้ container อยู่ภายใต้ network นั้นๆ

Cmdr

```
chalersak@Takeit-Youth:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
56b84a34bd66 nginx "/docker-entrypoint...." 8 minutes ago Up 8 minutes 80/tcp pensive_cray
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker network ls
NETWORK ID NAME DRIVER SCOPE
c096d77a1999 bridge bridge local
4a8130cfd284 host host local
05da587d0283 none null local
fec1dd152f0e testnetwork bridge local
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker network connect testnetwork pensive_cray
chalersak@Takeit-Youth:~$
```

# Docker Network ?

## Docker network Command

- `docker network disconnect <network name> <container name>` เพื่อ disconnect ให้ container อยู่ภายใต้ network นั้นๆ

```
Cmder
chalermsak@Takeit-Youth:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
56b84a34bd66 nginx "/docker-entrypoint...." 24 minutes ago Up 24 minutes 80/tcp pensive_cray
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker network ls
NETWORK ID NAME DRIVER SCOPE
c096d77a1999 bridge bridge local
4a8130cfd284 host host local
05da587d0283 none null local
fec1dd152f0e testnetwork bridge local
chalermsak@Takeit-Youth:~$
chalermsak@Takeit-Youth:~$ docker network disconnect testnetwork pensive_cray
chalermsak@Takeit-Youth:~$
```

# Docker Network ?

## Docker network Command

- `docker network rm <network name>` เพื่อลบ network ที่ถูกสร้างมาออก

```
Cmder
chalersak@Takeit-Youth:~$ docker network ls
NETWORK ID NAME DRIVER SCOPE
c096d77a1999 bridge bridge local
4a8130cfd284 host host local
05da587d0283 none null local
da0ea1e31968 testnetwork bridge local
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker network rm testnetwork
testnetwork
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker network ls
NETWORK ID NAME DRIVER SCOPE
c096d77a1999 bridge bridge local
4a8130cfd284 host host local
05da587d0283 none null local
chalersak@Takeit-Youth:~$ |
```

# Docker Network ?

## Create phpMyAdmin

- `docker run --name myadmin -d -e PMA_HOST=mariadb-container -p 7777:80 phpmyadmin`

```
Cmder
chalersak@Takeit-Youth:~$ docker run --name myadmin -d -e PMA_HOST=mysqldb -p 7777:80 phpmyadmin
Unable to find image 'phpmyadmin:latest' locally
latest: Pulling from library/phpmyadmin
f1f26f570256: Pull complete
ee0a4e40ccac: Pull complete
5ca9fb408faa: Pull complete
25862b797d87: Pull complete
5c128100b25a: Pull complete
c471030ba37f: Pull complete
285a5ec09eb5: Pull complete
c3223d84bfe5: Pull complete
bbd1867e192e: Pull complete
50aa2f131f8f: Pull complete
d8657c361c97: Pull complete
ee2a08bc4627: Pull complete
35cd0b63238c: Pull complete
522424f3fdee: Pull complete
e5a4a39ad7aa: Pull complete
1a9dbe91e3c5: Pull complete
87523e0bcadc: Pull complete
50e48d89125b: Pull complete
Digest: sha256:0ad74aed6f3f1c06d8b9fd936a8c7312069ccd72baf448f31dbb8891288bfa31
Status: Downloaded newer image for phpmyadmin:latest
5dc7ddd021a21e930e7d6246b47dafa266a30584ad63a3552d14f442a00803a6
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
5dc7ddd021a2 phpmyadmin "/docker-entrypoint..." 6 seconds ago Up 4 seconds 0.0.0.0:7777->80/tcp myadmin
3cf62f306ff4 mysql "docker-entrypoint.s..." 6 minutes ago Up 6 minutes 3306/tcp, 33060/tcp mysqldb
chalersak@Takeit-Youth:~$
```



# Docker Network ?

ทำให้ mariadb และ phpmyadmin เชื่อมต่อกันได้

- docker network create <network name>
- docker network connect <network name> <container name>

```
chalermsak@Takeit-Youth:~/network$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ace09068463d phpmyadmin "/docker-entrypoint..." About an hour ago Up About an hour 0.0.0.0:7777->80/tcp myadmin
18638189b8a9 mysql "docker-entrypoint.s..." About an hour ago Up About an hour 3306/tcp, 33060/tcp mysqlldb

chalermsak@Takeit-Youth:~/network$
chalermsak@Takeit-Youth:~/network$ docker network create network_app
f2826630dcd6249077dad008464dcf9a08f1d5c1bfced40a9267c413aa67d429
chalermsak@Takeit-Youth:~/network$
chalermsak@Takeit-Youth:~/network$ docker network ls
NETWORK ID NAME DRIVER SCOPE
011ba399310b bridge bridge local
5a1cdbaeab40 host host local
f2826630dcd6 network_app bridge local
636e1a9b1519 none null local

chalermsak@Takeit-Youth:~/network$
chalermsak@Takeit-Youth:~/network$
chalermsak@Takeit-Youth:~/network$ docker network connect network_app myadmin
chalermsak@Takeit-Youth:~/network$
chalermsak@Takeit-Youth:~/network$
chalermsak@Takeit-Youth:~/network$ docker network connect network_app mysqlldb
chalermsak@Takeit-Youth:~/network$
```

# Docker Network ?

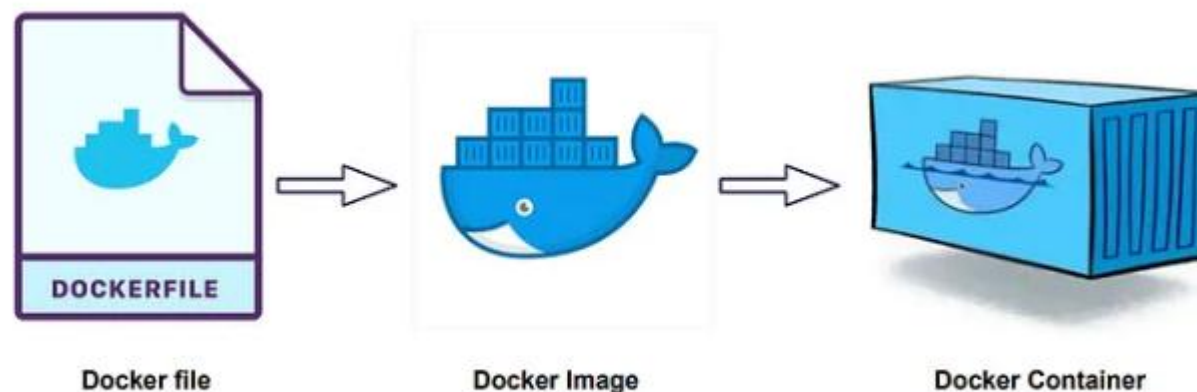
## Create pgadmin

- แบบฝึกหัด run container pgadmin โดยมีการใช้
  - โดยใช้ชื่อเป็น pgadmin-container
  - Port เป็น 8888:80
  - ใช้ environment เป็น -e PGADMIN\_DEFAULT\_EMAIL=admin@example.com  
-e PGADMIN\_DEFAULT\_PASSWORD=admin
  - ใส่ -d เพื่อ จะคืน cursor กลับมาด้วย
  - Image เป็น dpage/pgadmin4
  - Create network ชื่อ mypostgres-network
  - Inspect network เพื่อ check ว่าอยู่ใน network เดียวกันแล้วหรือยัง
  - Connect network ระหว่าง postgres database กับ pgadmin

# Dockerfile ?

## Dockerfile คืออะไร

Dockerfile คือ โค้ดคำสั่ง สำหรับสร้าง Docker Image เพื่อเป็นแม่แบบที่ใช้ในการสร้างเป็น Docker Container ซึ่งประกอบไปด้วยแอปพลิเคชันต่างๆ ที่จะทำงานเมื่อมีการเรียกใช้งานจาก Docker Container รวมทั้งการตั้งค่าจำลองสภาพแวดล้อม (Environment) ที่จำเป็นสำหรับการทำงานของ Container ไปด้วย



# Docker load & Docker Save

---

`docker save -o [ชื่อไฟล์] [ชื่อ image ที่จะทำการ save เป็นไฟล์]`

ex. `docker save -o my_image.zip my_image`

`docker load -i [ชื่อไฟล์ image ]`

ex. `docker load -i my_image.zip`

# Dockerfile ?

## Create Project angular

- สิ่งที่ต้องใช้ใน Dockerfile
  - FROM คือ การดึง image มาใช้เป็น base ในการ สร้าง
  - WORKDIR ใช้เพื่อกำหนด directory ที่จะถูกใช้เป็น working directory ภายในคอนเทนเนอร์
  - COPY ใช้เพื่อคัดลอกไฟล์หรือ directory จากไฟล์ระบบของโฮสต์ไปยังไฟล์ระบบภายในคอนเทนเนอร์
  - RUN ใช้เพื่อรันคำสั่งใดๆ ภายในคอนเทนเนอร์ในขั้นตอนการสร้างอิมเมจ
  - AS <builder> ใช้ร่วมกับคำสั่ง FROM เพื่อกำหนดชื่อให้กับแต่ละขั้นตอน
  - --from=<builder> เป็นคุณสมบัติที่สำคัญในการสร้างแบบหลายขั้นตอน (**multi-stage builds**) ซึ่งช่วยให้สามารถสร้างอิมเมจที่เล็กลงโดยใช้บางส่วนของ การสร้างอิมเมจจากขั้นตอนก่อนหน้านี้ (หรือจากอิมเมจอื่น)
  - EXPOSE ใช้เพื่อประกาศให้ Docker ทราบว่าคอนเทนเนอร์จะใช้พอร์ตใดในการรับส่งข้อมูล
  - CMD ใช้เพื่อระบุคำสั่งหรือโปรแกรมที่คอนเทนเนอร์จะรันเมื่อถูกสร้างขึ้น คำสั่งนี้จะถูกรันเมื่อคอนเทนเนอร์เริ่มทำงาน และจะถูกรันเพียงครั้งเดียว

# Dockerfile ?



---

## Create Dockerfile Stage 1

```
stage 1 build angular web
FROM node:18 AS build
WORKDIR /app
COPY . .
RUN npm i
RUN npm run build
```

# Dockerfile ?



---

## Create Dockerfile Stage 2

```
stage 2 deploy with nginx
FROM nginx:alpine
COPY --from=build /app/dist/01/browser /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

# Dockerfile ?

## Dockerfile Layers

Docker จะต้องจัดการ layers ที่มากขึ้น โดยแต่ละ Run จะสร้าง Layer ใหม่ขึ้นมา

ข้อดี

- จะสามารถเช็ค Debugging ง่ายกว่าถ้าเกิดปัญหา
- มีความยืดหยุ่นในการจัดการ สามารถจัดการแต่ละคำสั่งได้แยกกัน

ข้อเสีย

- ทำให้ขนาดใหญ่กว่า single Run

```
RUN apk update
RUN apk upgrade
RUN apk add python3
RUN apk add nodejs
RUN apk add git
```



# Dockerfile ?

## Dockerfile Layers

Single run

ข้อดี

- ช่วยลดจำนวน layer ที่ docker ตั้งจัดการและทำให้ image ขนาดเล็กลงด้วย
- ช่วยลดระยะเวลาในการสร้าง image
- ช่วยลดการสร้าง cache ซ้ำซ้อน

ข้อเสีย

- Debugging ยากขึ้นเมื่อเกิดปัญหา เพราะทุกคำสั่งอยู่ในบรรทัดเดียวกัน

```
RUN apk update && apk upgrade && \
 apk add --no-cache \
 python3 \
 nodejs \
 git
```

# Dockerfile ?

## Docker build

`docker build -t [name:tag] [path]`

- `-t [name:tag]`: ใช้เพื่อกำหนดชื่อและแท็ก (tag) ให้กับอิมเมจที่สร้างขึ้น โดย:
  - name คือชื่ออิมเมจ (อาจเป็นชื่อที่คุณตั้งเองหรือ repository ที่จะใช้)
  - tag คือเวอร์ชันหรือระบุเฉพาะเจาะจง (เช่น latest, v1.0, หรืออะไรก็ได้ตามต้องการ) โดยค่าเริ่มต้นถ้าไม่ระบุ tag จะใช้ latest
- `[path]`: ระบุตำแหน่ง directory ที่มี Dockerfile อยู่ (สามารถเป็น . ถ้าต้องการสร้างจาก directory ปัจจุบัน)

ตัวอย่าง

```
docker build -t my-web:0.0.1 .
```

# Dockerfile ?

---

## Run images

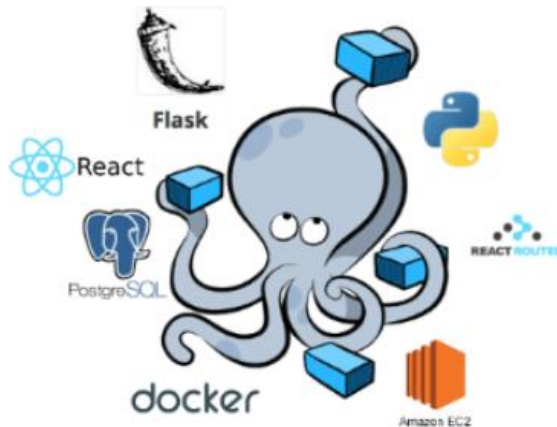
- Start container โดยใช้คำสั่ง

```
docker run --name web-container -d --rm -p 80:80 my-web
```

# Docker-compose ?

## Docker-compose คืออะไร

docker-compose คือ tool ที่ช่วยในการจัดการและรัน Docker Container หลายๆตัว ได้สะดวกมากยิ่งขึ้น โดยจะเขียนเป็นรูปแบบ YAML ไฟล์ที่ชื่อว่า docker-compose ขึ้นมา เพื่อ Config Application ในแต่ละตัวที่ต้องการสร้าง และสามารถใช้เพียงคำสั่งเดียวในการ สร้าง และ Start Service ทั้งหมด



# Docker-compose ?

## Docker-compose คืออะไร

- services คือ การจัดการ service ใน docker-compose.yml
- name service คือ ชื่อของ service ใช้ในการเรียกใช้ใน docker-compose.yml
- container\_name คือ การตั้งชื่อของ Container ที่จะสร้างขึ้น
- image คือ image ที่ถูกดึงมาใช้เป็น base ในการ Run
- build คือ การเรียกใช้ Dockerfile เพื่อสร้าง image
  - context: ระบุ directory ที่เก็บ Dockerfile
  - dockerfile: การระบุชื่อ Dockerfile
- environment คือ การกำหนดตัวแปรที่จะใช้ใน Container
- ports คือ การ expose port มาเพื่อให้เรียกใช้ Container จากข้างนอก
- volumes คือ การ map folder จากเครื่อง host เข้าไปใน Container
- network คือ กำหนดเครือข่ายที่ใช้เชื่อมต่อระหว่าง Container
- restart คือ กำหนดการ start ของ Container เมื่อการ Down
- depends\_on: คือ การระบุลำดับการ run service
- networks: คือ การสร้าง network
  - <network\_name> คือ ชื่อของ network ที่ตั้งขึ้นมาเพื่อเรียกใช้ใน docker-compose.yml
  - name: คือ ชื่อของ network เรียกใช้จากข้างนอก
  - driver: คือ การกำหนด ประเภทของ network

# Docker-compose ?



---

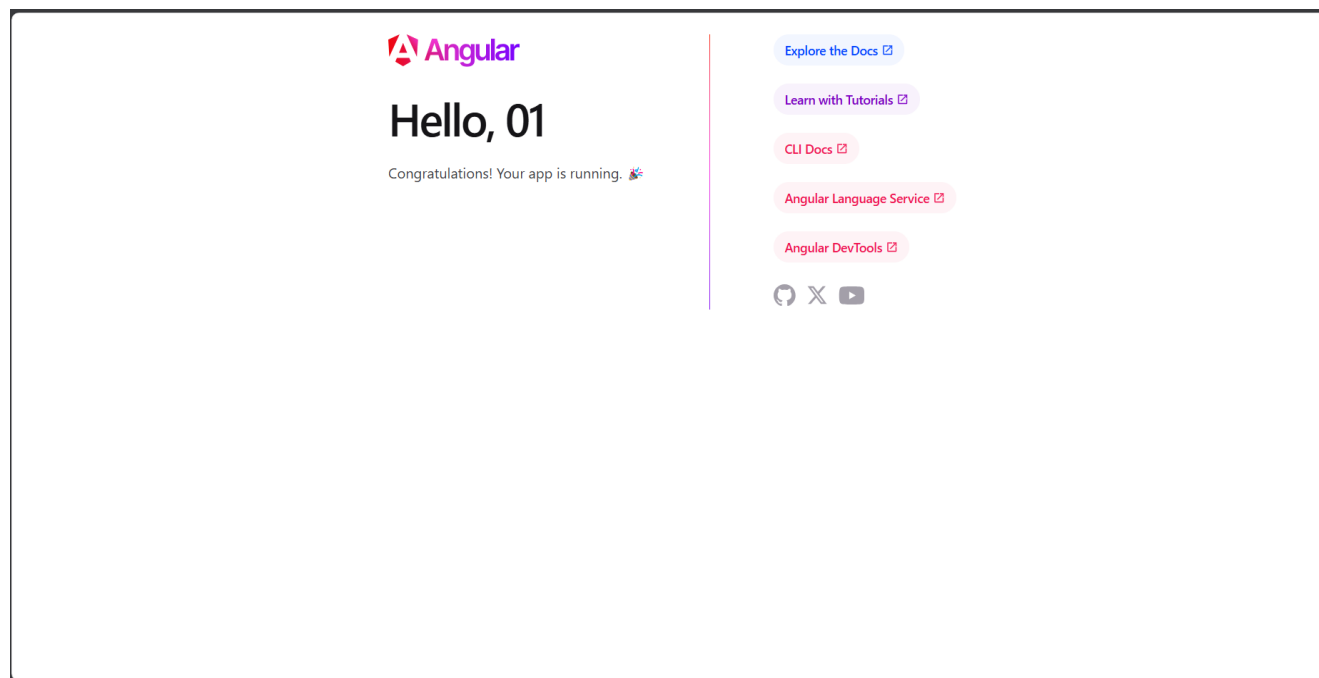
Create docker-compose.yml run angular

```
services:
 web:
 container_name: web
 build:
 context: .
 dockerfile: Dockerfile
 ports:
 - 8099:80
 environment:
 - TZ=Asia/Bangkok
```

# Dockerfile ?

## Run images

- เข้าไปที่ `http://localhost:8099`



# Docker registry ?

---

## Docker registry คืออะไร

**Docker Registry** ก็คือสถานที่เก็บและจัดการ Docker images หรือรูปแบบของโปรแกรมที่ถูกบรรจุใน container ง่ายๆ ๆ ก็คือเหมือนเป็นห้องสมุดที่เก็บหนังสือโปรแกรมให้เราเอาไปใช้ได้ตามต้องการ

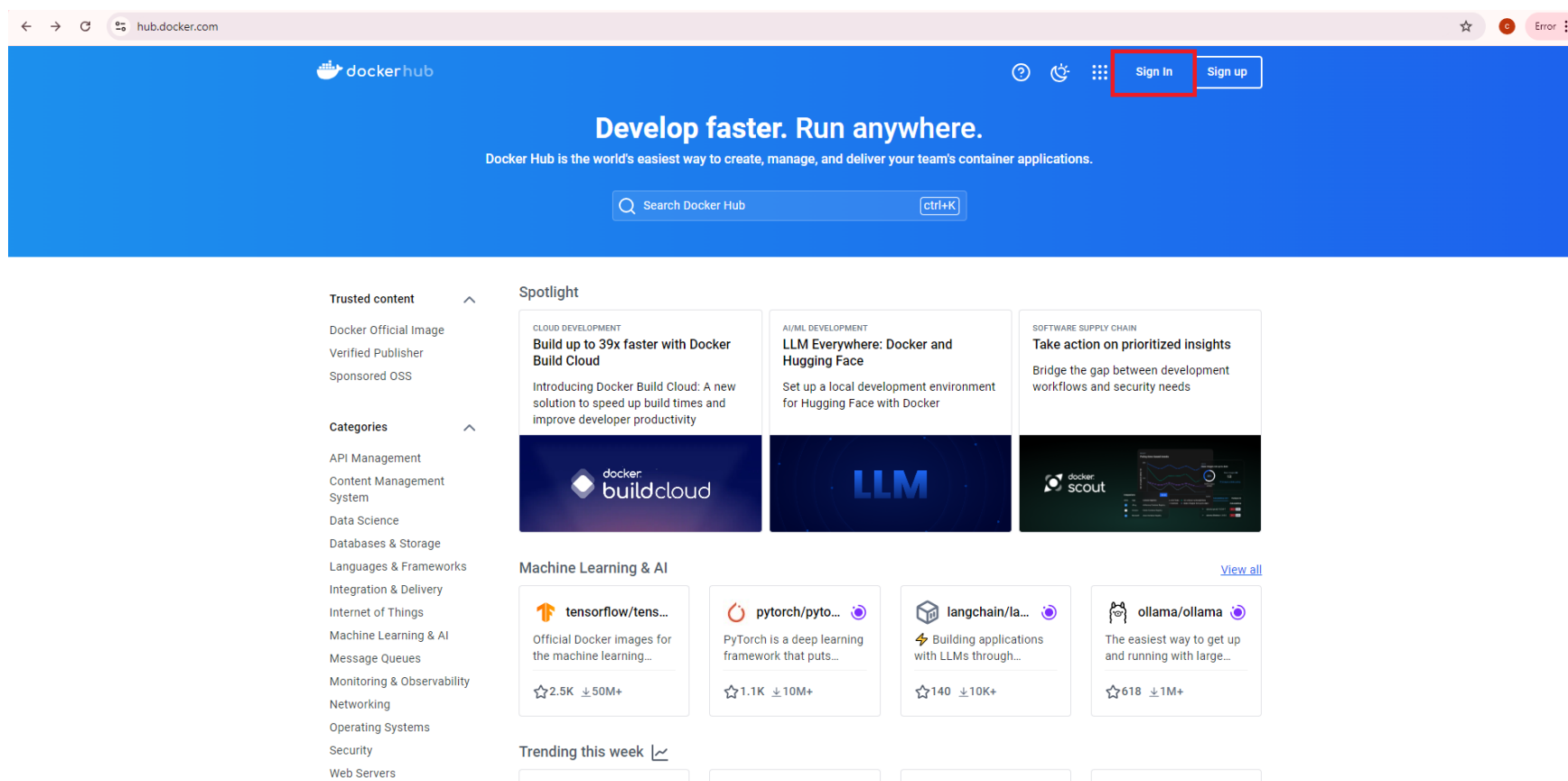
การทำงานหลัก ๆ ของ Docker Registry มีดังนี้

1. เก็บ **Docker images**: เก็บรูปแบบโปรแกรมที่อยู่ใน container
2. จัดการเวอร์ชัน: ทำให้เราสามารถใช่และเปลี่ยนเวอร์ชันของโปรแกรมได้สะดวก
3. กระจาย **Docker images**: ช่วยให้เราส่งและดึง Docker images ระหว่างเครื่องหรือระบบต่าง ๆ ได้ง่าย



# Docker registry ?

## Docker hub sign in



The screenshot shows the Docker Hub homepage. The browser address bar displays 'hub.docker.com'. The Docker Hub logo is in the top left. In the top right, there are icons for help, Docker Desktop, and a menu. The 'Sign In' button is highlighted with a red rectangle, and the 'Sign up' button is next to it. Below the navigation bar, the main heading reads 'Develop faster. Run anywhere.' followed by the tagline 'Docker Hub is the world's easiest way to create, manage, and deliver your team's container applications.' A search bar with the placeholder 'Search Docker Hub' and a 'ctrl+K' shortcut is centered. The left sidebar contains 'Trusted content' (Docker Official Image, Verified Publisher, Sponsored OSS) and 'Categories' (API Management, Content Management System, Data Science, Databases & Storage, Languages & Frameworks, Integration & Delivery, Internet of Things, Machine Learning & AI, Message Queues, Monitoring & Observability, Networking, Operating Systems, Security, Web Servers). The main content area features a 'Spotlight' section with three featured items: 'Build up to 39x faster with Docker Build Cloud', 'LLM Everywhere: Docker and Hugging Face', and 'Take action on prioritized insights'. Below this is a 'Machine Learning & AI' section with four featured Docker images: 'tensorflow/tensorflow', 'pytorch/pytorch', 'langchain/langchain', and 'ollama/ollama'. A 'Trending this week' section is partially visible at the bottom.

hub.docker.com

dockerhub

Sign In Sign up

Develop faster. Run anywhere.

Docker Hub is the world's easiest way to create, manage, and deliver your team's container applications.

Search Docker Hub ctrl+K

Trusted content

- Docker Official Image
- Verified Publisher
- Sponsored OSS

Categories

- API Management
- Content Management System
- Data Science
- Databases & Storage
- Languages & Frameworks
- Integration & Delivery
- Internet of Things
- Machine Learning & AI
- Message Queues
- Monitoring & Observability
- Networking
- Operating Systems
- Security
- Web Servers

Spotlight

CLOUD DEVELOPMENT

**Build up to 39x faster with Docker Build Cloud**

Introducing Docker Build Cloud: A new solution to speed up build times and improve developer productivity

AI/ML DEVELOPMENT

**LLM Everywhere: Docker and Hugging Face**

Set up a local development environment for Hugging Face with Docker

SOFTWARE SUPPLY CHAIN

**Take action on prioritized insights**

Bridge the gap between development workflows and security needs

docker buildcloud

LLM

docker scout

Machine Learning & AI

View all

tensorflow/tensorflow

Official Docker images for the machine learning...

☆2.5K ±50M+

pytorch/pytorch

PyTorch is a deep learning framework that puts...

☆1.1K ±10M+

langchain/langchain

Building applications with LLMs through...

☆140 ±10K+

ollama/ollama

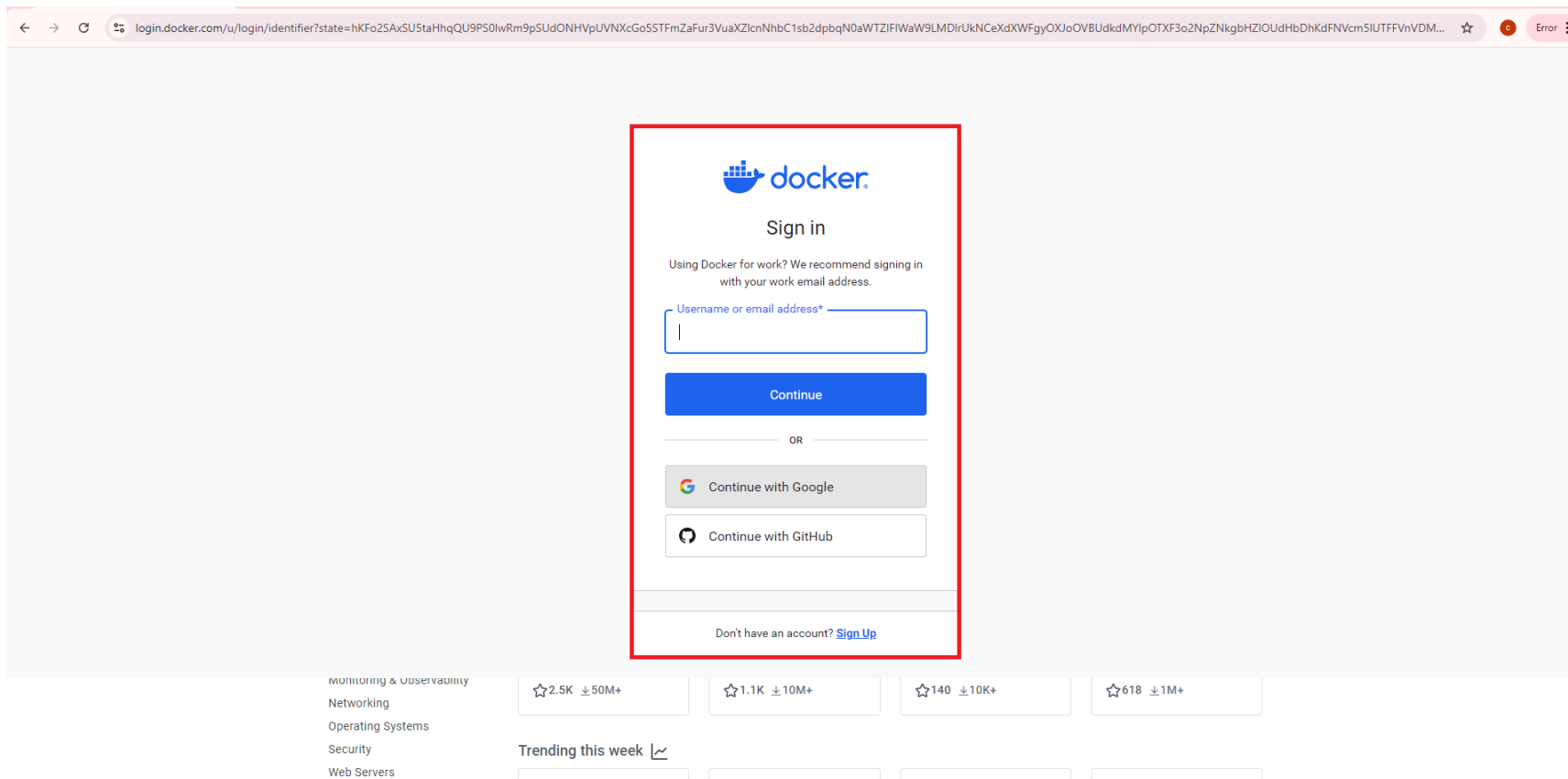
The easiest way to get up and running with large...

☆618 ±1M+

Trending this week

# Docker registry ?

## Docker hub sign in Gmail



The screenshot shows the Docker Hub login page. A red rectangular box highlights the central sign-in form. The form includes the Docker logo, a 'Sign in' heading, a message recommending work email login, a text input field for 'Username or email address\*', a blue 'Continue' button, and social login options for Google and GitHub. Below the form is a 'Sign Up' link for new users. The browser's address bar shows a complex state parameter. The footer contains category links and trending items.

login.docker.com/u/login/identifier?state=hKfo2SAx5U5taHhQQU9PS0lwRm9pSudONHVpUVNXcGo5STFmZaFur3YuaXZlcnNhbC1sb2dpbqN0aWZlZlFIW9LMDlrUkNceXdXWFgyOXJoOVBUdkdMYlpOTXF3o2NpZnkgbHZlOUdHbDhKdFNVcm5lUTFFVnVDM...

**docker.**

Sign in

Using Docker for work? We recommend signing in with your work email address.

Username or email address\*

Continue

OR

Continue with Google

Continue with GitHub

Don't have an account? [Sign Up](#)

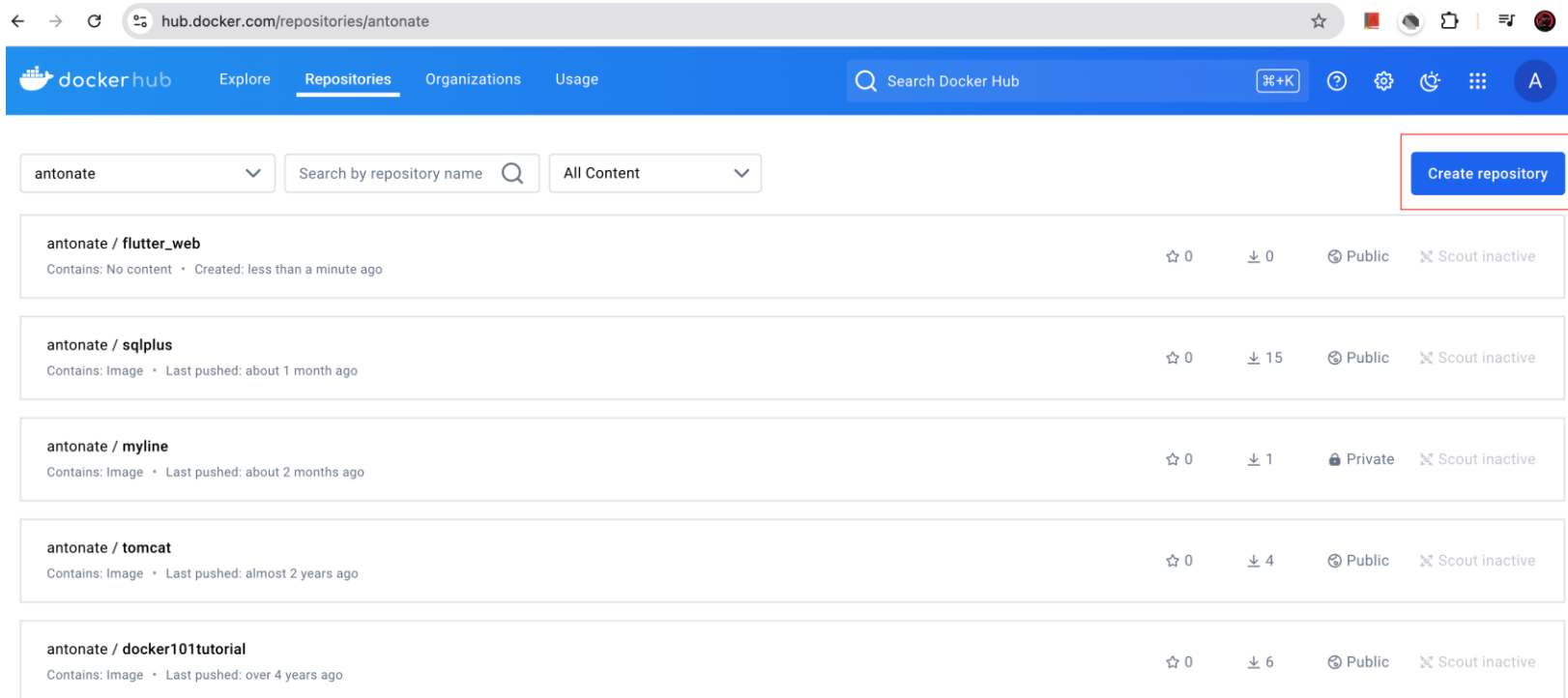
monitoring & observability  
Networking  
Operating Systems  
Security  
Web Servers

☆2.5K ±50M+   ☆1.1K ±10M+   ☆140 ±10K+   ☆618 ±1M+

Trending this week ↗

# Docker registry ?

## Docker hub Create repository



The screenshot shows the Docker Hub interface for the user 'antonate'. The browser address bar displays 'hub.docker.com/repositories/antonate'. The navigation bar includes 'dockerhub', 'Explore', 'Repositories' (active), 'Organizations', and 'Usage'. A search bar is present with the text 'Search Docker Hub'. Below the navigation bar, there is a filter section with a dropdown menu set to 'antonate', a search input field with the placeholder 'Search by repository name', and another dropdown menu set to 'All Content'. A red rectangular box highlights the 'Create repository' button in the top right corner of the repository list.

| Repository                                                                       | Stars | Downloads | Visibility | Scout Status   |
|----------------------------------------------------------------------------------|-------|-----------|------------|----------------|
| antonate / flutter_web<br>Contains: No content • Created: less than a minute ago | 0     | 0         | Public     | Scout inactive |
| antonate / sqlplus<br>Contains: Image • Last pushed: about 1 month ago           | 0     | 15        | Public     | Scout inactive |
| antonate / myline<br>Contains: Image • Last pushed: about 2 months ago           | 0     | 1         | Private    | Scout inactive |
| antonate / tomcat<br>Contains: Image • Last pushed: almost 2 years ago           | 0     | 4         | Public     | Scout inactive |
| antonate / docker101tutorial<br>Contains: Image • Last pushed: over 4 years ago  | 0     | 6         | Public     | Scout inactive |

# Docker registry ?

## Docker hub Create repository

[Repositories](#) / [Create](#)

Using 1 of 1 private repositories.

### Create repository

Namespace

antonate

Repository Name \*

flutter\_web



Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

### Visibility

Using 1 of 1 private repositories. [Get more](#)



Public

Appears in Docker Hub search results



Private

Only visible to you

### Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.

Cancel

Create

# Docker registry ?

## Docker hub repository

antonate / [Repositories](#) / [flutter\\_web](#) / [General](#)

Using 1 of 1 private repositories.

[General](#) [Tags](#) [Builds](#) [Collaborators](#) [Webhooks](#) [Settings](#)

antonate/flutter\_web 

Created 3 minutes ago

This repository does not have a description   INCOMPLETE

This repository does not have a category   INCOMPLETE

### Docker commands

To push a new tag to this repository:

[Public View](#)

```
docker push antonate/flutter_web:tagname
```

Tags  INCOMPLETE

This repository is empty. Push some images to it to see them appear here.

### Automated Builds

Manually pushing images to Hub? Connect your account to GitHub or Bitbucket to automatically build and tag new images whenever your code is updated, so you can focus your time on creating.

Available with Pro, Team and Business subscriptions. [Read more about automated builds](#) .

[Upgrade](#)

Repository overview  INCOMPLETE

An overview describes what your image does and how to run it. It displays in [the public view of your repository](#) once you have pushed some content.

# Docker registry ?

## Login Docker Hub

- `docker login -u <Gmail> -p <Password>`

```
chalersak@Takeit-Youth:~/myproject/angular$ docker login -u [REDACTED] -p [REDACTED]
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
Login Succeeded
chalersak@Takeit-Youth:~/myproject/angular$
chalersak@Takeit-Youth:~/myproject/angular$
chalersak@Takeit-Youth:~/myproject/angular$
chalersak@Takeit-Youth:~/myproject/angular$ |
```

# Docker registry ?

## Create new images

- `docker tag <image> <new-image>`

```
root@a0fe26d667c3:/workspaces/web_project# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
flutter_web latest 758281d0a68f 26 minutes ago 63.4MB
<none> <none> 37f7e178d90c 2 hours ago 63.2MB
<none> <none> 2eeea20018a3 2 hours ago 3.11GB
<none> <none> c02b4303a2f4 2 hours ago 3.11GB
<none> <none> edb45074982a 2 hours ago 3.18GB
flutter_devcon latest bddee0f40657 7 hours ago 3.01GB
dpage/pgadmin4 latest 69eb871cd151 6 days ago 497MB
phpmyadmin latest edc4c5900fdc 3 weeks ago 572MB
mariadb latest 34094fbe018c 3 weeks ago 441MB
ubuntu latest 2b1b17d5e5a2 4 weeks ago 101MB
nginx latest 195245f0c792 6 weeks ago 193MB
postgres latest 75282fa229a1 6 weeks ago 453MB
mysql latest 4e8a34aea708 2 months ago 609MB
busybox latest 7db2ddde018a 16 months ago 4.04MB

root@a0fe26d667c3:/workspaces/web_project# docker tag flutter_web:latest antonate/flutter_web:latest
root@a0fe26d667c3:/workspaces/web_project# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
antonate/flutter_web latest 758281d0a68f 27 minutes ago 63.4MB
flutter_web latest 758281d0a68f 27 minutes ago 63.4MB
<none> <none> 37f7e178d90c 2 hours ago 63.2MB
<none> <none> 2eeea20018a3 2 hours ago 3.11GB
<none> <none> c02b4303a2f4 2 hours ago 3.11GB
<none> <none> edb45074982a 2 hours ago 3.18GB
flutter_devcon latest bddee0f40657 7 hours ago 3.01GB
dpage/pgadmin4 latest 69eb871cd151 6 days ago 497MB
phpmyadmin latest edc4c5900fdc 3 weeks ago 572MB
mariadb latest 34094fbe018c 3 weeks ago 441MB
ubuntu latest 2b1b17d5e5a2 4 weeks ago 101MB
nginx latest 195245f0c792 6 weeks ago 193MB
postgres latest 75282fa229a1 6 weeks ago 453MB
mysql latest 4e8a34aea708 2 months ago 609MB
busybox latest 7db2ddde018a 16 months ago 4.04MB
```

# Docker registry ?

## Docker push images to Docker Hub

- `docker push <new-image:tag>`

```
● root@a0fe26d667c3:/workspaces/web_project# docker push antonate/flutter_web:latest
The push refers to repository [docker.io/antonate/flutter_web]
c24721c9fe22: Pushed
9920da5a42fa: Mounted from library/nginx
bf283905f539: Mounted from library/nginx
73d220ae8858: Mounted from library/nginx
583072717943: Mounted from library/nginx
b281b033c834: Mounted from library/nginx
bc0dbe0c4fef: Mounted from library/nginx
edf70074bd40: Mounted from library/nginx
latest: digest: sha256:6886b3fac4a16ed73f7511358ec0dd32e30187602347df7f89dbbbbd1287ff372 size: 1992
○ root@a0fe26d667c3:/workspaces/web_project#
```



# Docker registry ?

## Docker Hub

 **dockerhub** [Explore](#) [Repositories](#) [Organizations](#) [Usage](#)       

antonate / [Repositories](#) / [flutter\\_web](#) / [Tags](#) Using 1 of 1 private repositories.

[General](#) [Tags](#) [Builds](#) [Collaborators](#) [Webhooks](#) [Settings](#)

☐ Sort by 

Newest ▾

Filter Tags 

Delete

☐

TAG

 [latest](#)

Last pushed a few seconds ago by [antonate](#)

Digest

[6886b3fac4a1](#)

OS/ARCH

linux/arm64/v8

Last pull

a few seconds ago

Compressed Size ⓘ

22.47 MB

`docker pull antonate/flutter_web:latest`

Copy

# Docker registry ?

## Docker pull image Docker Hub

- `docker pull <new-image:tag>`

```
busybox latest 7db2ddde618a 10 months ago 4.04MB
● root@a0fe26d667c3:/workspaces/web_project# docker pull antonate/flutter_web
Using default tag: latest
latest: Pulling from antonate/flutter_web
Digest: sha256:6886b3fac4a16ed73f7511358ec0dd32e30187602347df7f89dbbbd1287ff372
Status: Image is up to date for antonate/flutter_web:latest
docker.io/antonate/flutter_web:latest
○ root@a0fe26d667c3:/workspaces/web_project# █
```

# Docker system ?



---

## Docker system คืออะไร

Docker system คือชุดคำสั่งที่ใช้ในการจัดการระบบ Docker ซึ่งช่วยให้เราสามารถดูแลและจัดการทรัพยากรต่างๆ ที่ Docker ใช้งานอยู่ได้อย่างมีประสิทธิภาพ

# Docker system ?

## Docker system command

- docker system df ใช้สำหรับตรวจสอบทรัพยากร ว่ามีการใช้งานไปจำนวนเท่าไรแล้ว

```
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ docker system df
TYPE TOTAL ACTIVE SIZE RECLAIMABLE
Images 0 0 0B 0B
Containers 0 0 0B 0B
Local Volumes 0 0 0B 0B
Build Cache 0 0 0B 0B
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$
chalersak@Takeit-Youth:~$ |
```

# Docker system ?

## Docker system command

- docker system events ใช้สำหรับติดตามเหตุการณ์ต่างๆ ที่เกิดขึ้นในระบบ Docker แบบ Realtime เช่นการ start หรือ stop Container

```
chalermsak@Takeit-Youth:~/stack$ docker system events
```

```
2024-08-01T10:06:13.748469066+07:00 image tag sha256:6fb315526c9eb408e6e14a1c592e129c9f0429946313bb3a3aac18bc1e3648ac (com.docker.compose.project=angular, com
.docker.compose.service=angular, com.docker.compose.version=2.27.1, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=multi-stage:1.0.0)
2024-08-01T10:06:13.971744723+07:00 network create 86107e90b7b47c67d870ba6b2d8af3bd232670a4c634a42e67cb4e55cb150cbf (name=angular_default, type=bridge)
2024-08-01T10:06:14.032856112+07:00 container create 2c522b79545313be6da5381f95240d0e6f7e8c26020305fee1792cf0945c86a1 (com.docker.compose.config-hash=6dc96c46
18a19d1903e7e9af235e08f0064f39f752d9db68797545bfd4336df9, com.docker.compose.container-number=1, com.docker.compose.depends_on=, com.docker.compose.image=sha2
56:6fb315526c9eb408e6e14a1c592e129c9f0429946313bb3a3aac18bc1e3648ac, com.docker.compose.oneoff=False, com.docker.compose.project=angular, com.docker.compose.p
roject.config_files=/home/chalermsak/myproject/angular/docker-compose.yml, com.docker.compose.project.working_dir=/home/chalermsak/myproject/angular, com.dock
er.compose.service=angular, com.docker.compose.version=2.27.1, desktop.docker.io/wsl-distro=Ubuntu-20.04, image=multi-stage:1.0.0, maintainer=NGINX Docker Mai
ntainers <docker-maint@nginx.com>, name=angular_multi-stage)
2024-08-01T10:06:14.115556422+07:00 network connect 86107e90b7b47c67d870ba6b2d8af3bd232670a4c634a42e67cb4e55cb150cbf (container=2c522b79545313be6da5381f95240d
0e6f7e8c26020305fee1792cf0945c86a1, name=angular_default, type=bridge)
2024-08-01T10:06:14.415585107+07:00 container start 2c522b79545313be6da5381f95240d0e6f7e8c26020305fee1792cf0945c86a1 (com.docker.compose.config-hash=6dc96c461
8a19d1903e7e9af235e08f0064f39f752d9db68797545bfd4336df9, com.docker.compose.container-number=1, com.docker.compose.depends_on=, com.docker.compose.image=sha25
6:6fb315526c9eb408e6e14a1c592e129c9f0429946313bb3a3aac18bc1e3648ac, com.docker.compose.oneoff=False, com.docker.compose.project=angular, com.docker.compose.pr
oject.config_files=/home/chalermsak/myproject/angular/docker-compose.yml, com.docker.compose.project.working_dir=/home/chalermsak/myproject/angular, com.dock
er.compose.service=angular, com.docker.compose.version=2.27.1, desktop.docker.io/wsl-distro=Ubuntu-20.04, image=multi-stage:1.0.0, maintainer=NGINX Docker Main
tainers <docker-maint@nginx.com>, name=angular_multi-stage)
```

Use `docker system events` to get real-time events from the server. These events differ per Docker object type.

Docker container

Different event types have different scopes. Local-scoped events are only seen on the node they take place

# Docker system ?

## Docker system command

- docker system info ใช้สำหรับดูข้อมูลทั้งหมดเกี่ยวกับ Docker เช่น docker version , container ที่ทำงานอยู่ , images ที่มีในระบบ เป็นต้น

```
chalersak@Takeit-Youth:~$ docker system info
Client:
Version: 26.1.4
Context: default
Debug Mode: false
Plugins:
buildx: Docker Buildx (Docker Inc.)
 Version: v0.14.1-desktop.1
 Path: /usr/local/lib/docker/cli-plugins/docker-buildx
compose: Docker Compose (Docker Inc.)
 Version: v2.27.1-desktop.1
 Path: /usr/local/lib/docker/cli-plugins/docker-compose
debug: Get a shell into any image or container (Docker Inc.)
 Version: 0.0.32
 Path: /usr/local/lib/docker/cli-plugins/docker-debug
dev: Docker Dev Environments (Docker Inc.)
 Version: v0.1.2
 Path: /usr/local/lib/docker/cli-plugins/docker-dev
extension: Manages Docker extensions (Docker Inc.)
 Version: v0.2.24
 Path: /usr/local/lib/docker/cli-plugins/docker-extension
feedback: Provide feedback, right in your terminal! (Docker Inc.)
 Version: v1.0.5
 Path: /usr/local/lib/docker/cli-plugins/docker-feedback
init: Creates Docker-related starter files for your project (Docker Inc.)
 Version: v1.2.0
 Path: /usr/local/lib/docker/cli-plugins/docker-init
sbom: View the packaged-based Software Bill Of Materials (SBOM) for an image (Anchore Inc.)
 Version: 0.6.0
 Path: /usr/local/lib/docker/cli-plugins/docker-sbom
scout: Docker Scout (Docker Inc.)
 Version: v1.9.3
 Path: /usr/local/lib/docker/cli-plugins/docker-scout
Server:
Containers: 1
Running: 1
Paused: 0
Stopped: 0
Images: 2
```

# Docker system ?

## Docker system command

- docker system prune ใช้สำหรับลบสิ่งที่ไม่ใช้งานแล้วเพื่อคืนพื้นที่ disk เช่น container , images , volume และ network ที่ไม่ได้ใช้แล้ว

```
chalersak@Takeit-Youth:~/myproject/angular$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
multi-stage 1.0.0 1b1a473edc81 2 hours ago 43.1MB
node 22.5.1-alpine 5c4cc5767575 12 days ago 151MB
chalersak@Takeit-Youth:~/myproject/angular$
chalersak@Takeit-Youth:~/myproject/angular$ docker system prune -a --volumes -f
Deleted Images:
untagged: multi-stage:1.0.0
deleted: sha256:1b1a473edc81372c37ab16c115e27fac62c7d9be803e702a7e0b01f16a7aefc4
untagged: node:22.5.1-alpine
untagged: node@sha256:9fcc1a6da2b9eee38638df75c5f826e06e9c79f6a0f97f16ed98fe0ebb0725c0
deleted: sha256:5c4cc5767575c711b99b1b077ad8afb8c7bf407aca174b7cbc998ce6db1e4f93
deleted: sha256:039eb2ff3cb619a6ab2a6457aa744c39b4533c9ed16823c645b665d6c8eb575a
deleted: sha256:63d1c731ac34ca86bb0e2bb4fb612ab5828c1d449f2b8f2711f0bf007687ee07
deleted: sha256:4b5d1edca6e98bc185b679164cc7a68b668a2127fb1ecd9aab657410d202d34e
deleted: sha256:78561cef0761903dd2f7d09856150a6d4fb48967a8f113f3e33d79effbf59a07
```

```
chalersak@Takeit-Youth:~/myproject/angular$
chalersak@Takeit-Youth:~/myproject/angular$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
chalersak@Takeit-Youth:~/myproject/angular$
chalersak@Takeit-Youth:~/myproject/angular$
chalersak@Takeit-Youth:~/myproject/angular$
```

แบบประเมิน

